

密级:_____



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

针对 AES 加密算法的缓存攻击研究

作者姓名: _____ 唐烨

指导教师: _____ 李慧云 研究员

_____ 中国科学院深圳先进技术研究院

学位类别: _____ 工学硕士

学科专业: _____ 信号与信息处理

研究所: _____ 中国科学院深圳先进技术研究院

2016 年 4 月

Research of Cache Attack on AES Encryption Algorithm

**By
Ye Tang**

**A Dissertation/Thesis Submitted to
The University of Chinese Academy of Sciences
In partial fulfillment of the requirement
For the degree of
Master of Science in Engineering**

**Shenzhen Institutes of Advanced Technology, Chinese Academy of
Sciences**

April, 2016

声 明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：

日期：

摘 要

随着互联网的高速发展，我们驶上了一条信息化的高速公路。与此同时，通信活动中的信息安全问题，以惊人的速度快速增长。缓存攻击（Cache Attack）是一种针对密码算法的侧信道攻击手段，它基于高速缓存的行为特征展开攻击，这种特征被称为缓存的命中与失效。由于缓存攻击的效果显著且容易实现，它成为了信息安全分析手段中不可或缺的一种技术。

本文提出了一种结合缓存攻击的侧信道攻击模型，它的目标是诸 AES(Advanced Encryption Standard) 的对称加密算法。模型是在该类型其他攻击手段失效的研究现状下提出的，并且首次从数学上对该类型攻击的效率做了量化。这个模型利用了分组加密算法在密码分块链接(CBC, Cipher-Block Chaining)工作模式下的一个漏洞----Padding Oracle 漏洞，该漏洞属于 CCA (Chosen Ciphertext Attack) 安全缺陷，因为如果攻击者具有使用密码算法解密任何密文的能力，则漏洞就会出现在算法中。攻击模型中的缓存攻击模块，解决了当前利用 Padding Oracle 漏洞的方法中无法获取到 Oracle 这一难题，而 Oracle 信息对展开攻击是至关重要的。本文提出的攻击模型，通过监测与产生 Oracle 的相关指令，然后分析加解密的算法流程，就推断出有用的信息，最后恢复成密文所对应的明文。

使用攻击模型，本文进行了密文单字节的攻击恢复实验。实验在单台计算机和跨虚拟机的两种攻击场景下进行，攻击目标为 GnuTLS 安全库中的密文解密部分。单台计算机下的实验结果显示，单个密文的恢复成功率可以达到 99.5%，高于该研究领域中最新攻击方法的攻击效率。跨虚拟机下的攻击实验中，通过构造重复密文的方式，也可以实现恢复明文的目的。两种场景下的实验都证明了本文所提出的攻击模型的有效性，文中给出了抵抗这种攻击的相关对策。

关键词：密码学；缓存攻击；AES；Padding Oracle 攻击；侧信道；CCA 安全；

Research of Cache Attack on AES Encryption Algorithm

Ye Tang (Information Security)

Directed by Huiyun Li

Along with the fast development of the international Internet, we are moving on the information superhighway. But the security issues during the communicating actives are growing at an astronomical rate, which attract increasing attention in recent years. Cache attack is one of side-channel attacks applied in cryptanalysis, it exploits the various features of Cache operation results, namely Cache-hit and Cache-miss. Because of the high attack efficiency and the simplicity, Cache attack has become indispensable in security information analysis.

This paper proposes a side-channel attack model based on Cache attack, against symmetric encryption algorithms, such as AES (Advanced Encryption Standard). This model is brought forward in such research background that the same type of attack methods is invalid, and it is the first to quantify the attack efficiency mathematically. This attack model leverages the Padding Oracle weakness, which is associated with security defects in block cipher algorithms operating in CBC (Cipher-Block Chaining) mode. This flaw is one of CCA (Chosen Ciphertext Attack) security vulnerabilities because the cryptographic scheme using CBC is weak if an attacker has the ability to decrypt any ciphertext. The Cache attack module solves the key problems in current Padding Oracle attack: it may be impossible to obtain the oracle message which is crucial to the attack success. The attack model proposed in this paper can monitor the execution flow of the instructions related to generating the Oracle generation. Then useful information can be inferred by analyzing detailed algorithm process. At last, some or even all of plaintext can be recovered directly

Attack experiment targeting single byte of ciphertext to recover plaintext was conducted in two scenarios: a single machine and cross-VMs (Virtual Machine), aiming at cipher decrypting in GnuTLS (a famous open source library). The result of the single machine experiment revealed that success rate for recover one bite could reach 99.5%, which is more efficient than the latest attack. Meanwhile cross-VMs experiment showed that plaintext could also be recovered with higher success rate by constructing complicated ciphertext chain. These scenarios experiment demonstrated the validity and high-efficiency of the attack model. In the last part, several countermeasures are suggested to resist attacks based on the attack model.

Keywords: Cryptography, Cache Attack, AES, Padding Oracle, Side-Channel, CCA Security

目 录

第一章 引言	1
1.1 研究背景和意义	1
1.2 国内外研究现状	2
1.2.1 缓存（Cache）攻击	2
1.2.2 Padding Oracle 漏洞	3
1.3 论文研究的内容	4
1.3.1 论文的研究思路	4
1.3.2 论文的主要贡献	5
1.4 论文的组织	5
第二章 数学理论	7
2.1 AES 加密算法原理	7
2.1.1 AES 算法的工作流程	7
2.1.2 AES 算法的加密操作模式	8
2.2 AES 算法的安全性	11
2.2.1 对称密码方案的安全性	11
2.2.2 选择明文攻击（CPA）	12
2.2.3 选择密文攻击（CCA）	13
2.3 本章小结	15
第三章 攻击基础	16
3.1 缓存攻击原理	16
3.1.1 缓存攻击基础	16
3.1.3 缓存攻击实例	17
3.2 数据报传输层安全协议	20
3.2.1 DTLS 协议概述	20
3.2.2 DTLS 会话安全	20
3.2.3 常见的开源库实现	21
3.3 Padding Oracle 漏洞	22
3.3.1 漏洞原理概述	22
3.3.2 最新的漏洞构造	23
3.4 本章小结	24
第四章 模型建立	25
4.1 CCA 攻击实验	25

4.2 缓存攻击模型	28
4.2.1 加解密模块	29
4.2.2 数据恢复模块	30
4.2.3 缓存攻击模块	30
4.3 单台计算机攻击实验	33
4.3.1 功能模块划分	33
4.3.2 确定攻击位置	34
4.3.3 调整环境参数	35
4.4 跨虚拟机攻击实验	35
4.5 攻击效率分析	36
4.6 本章小结	38
第五章 实验结果	39
5.1 环境参数	39
5.2 攻击实验结果	40
5.2.1 缓存行为特征实验结果	40
5.2.2 单台计算机攻击实验结果	40
5.2.3 跨虚拟机攻击实验结果	41
5.3 实验结果分析	43
5.4 抵抗攻击的措施	44
5.5 本章小结	44
第六章 总结与展望	47
6.1 研究总结	47
6.2 研究展望	48

图目录

图 2.1	AES 算法单轮加密状态变化	8
图 2.2	电子密码本链接模式 (ECB)	9
图 2.3	密码分组链接模式 (CBC)	9
图 2.4	计数器链接模式 (CTR)	10
图 2.5	输出反馈链接模式 (OFB)	10
图 2.6	密码反馈链接模式 (CFB)	11
图 2.7	MAC 算法两种不同的使用顺序	14
图 3.1	缓存的层次结构	16
图 3.2	缓存攻击检测算法执行流	18
图 3.3	DTLS 协议中的加解密策略	21
图 4.1	攻击者 A 恢复明文的过程	27
图 4.2	基于 CCA 缺陷的缓存攻击模型	29
图 4.3	缓存攻击模块核心流程	33
图 4.4	攻击进程和目标进程的进程空间分布	34
图 4.5	单台计算机和跨虚拟机两种情形中进程的位置	36
图 5.1	缓存的行为特征	40
图 5.2	单台计算机上单字节攻击结果	41
图 5.3	跨虚拟机单字节攻击结果	42
图 5.4	跨虚拟机单字节攻击成功率	43

表目录

表格 3.1	缓存攻击的算法描述	19
表格 4.1	数据恢复模块的算法描述	30
表格 4.2	GnuTLS 中解密代码片段	32
表格 5.1	攻击实验的环境参数	39

第一章 引言

本章首先介绍了研究工作的背景及意义，以及目前在该领域，国内外发展现状。接下来的一节概述了本文所提出方法的研究思路，以及这种攻击方法的创新点和贡献。最后一节是本论文的章节安排。

1.1 研究背景和意义

在信息时代飞速演进的今天，大量信息以数字形式存放在信息系统中，信息的传输则是在开放、不安全的公共信道上。而这些信息系统和公共信道在没有设防的情况下是非常脆弱的，很容易受到入侵者的攻击和破坏。如何保护公共网络上信息的安全，已经受到了越来越多人的关注。密码学作为信息安全的核心之一，其重要性也与日俱增，吸引大量的研究人员投入到密码技术的分析与研究中。

密码学的研究主要分为两个部分：密码编码学和密码分析学^[1]。密码编码学的研究目标是设计或改进更为安全的加解密算法，以保护通信过程中的数据安全；而密码分析学的目标不同，它主要通过分析手段，破解密码算法或获取算法密钥，达到信息窃取、解读和利用的目的。两者之间既相互对立又相互促进，一方面，针对已有的密码分析手段，密码编码者总希望设计出可以抵抗所有已知分析的密码算法，密码分析者希望发现密码算法存在的安全缺陷，密码分析方法的进步反馈了密码算法的设计工作，它为后者提供了丰富的分析思路和研究动力。两个研究方向相互促进，共同对密码学的发展起到了不可取代的作用。

在已有的密码分析方法中，有一类被称为侧信道攻击^[2]。它利用的是密码学中这样一个事实：密码算法设计的理论安全性并不等于其在设备上物理实现的安全性^[3]。虽然可以将密码算法视为一个黑盒，这个黑盒接收明文或密文的输入，对应输出密文或明文。但是这个黑盒在运行的过程中，可能会有光线、声音、或者其他物理量（温度、电磁强度、执行时间、电压、电流等）的变化，这些变化为密码分析引入了新的参考。从信息论的角度来考虑，这些侧信道信息降低了加密算法在明文上增加的随机性，使得这些密文变得不再安全。侧信道攻击被提出来之初，主要的应用目标是在电子设备上，因为电子设备在运行加解密这种资源消耗较大的操作时，对外会有功率或者电磁的差异性变化，攻击者收集到这些信息后进行分析，从而可以得到密钥等敏感信息。近年来，随着研究的拓展，侧信道技术利用的有用信息，逐渐地从设备外部深入到计算机内部的CPU、高速缓存（Cache）、分支预测单元等上面，信息的获取途径更为多样化。互联网的发展同时也给侧信道技术的发展带来了新的契机，攻击者使用诸如时间侧信道的方法，可以得到网络通信中的私密数据。

新的密码分析方法（如侧信道技术）的出现，对现有的加密设备、互联网应用、安

全协议等来了新的挑战。同时,也会推动密码算法设计者、安全协议制定者和信息安全公司相应地采取新的防御手段。如文献^[4]提出的差分功耗攻击技术,可以通过统计 DES 加密算法运行时的功耗,进行差分计算,即可获取算法在加密过程中使用的密钥。针对这种技术,已经厂商推出了抗功耗的安全芯片。又如 2014 年 4 月被公布的 OpenSSL 的 Heartbleed 漏洞^[5],使全球三分之二的网站受到影响,千千万万的服务器处于危险。该漏洞被提出后不久,相关开源库的维护人员就给出了相应的安全修复措施。

通过探寻新的密码分析方法,找到密码算法在实践中的安全隐患,并在这些弱点上建立一系列有效的物理或算法防护手段,从而使得信息不被窃取或者免于篡改,这有利于信息的传播安全。同时,新的密码分析方法的提出,还可以增强广大计算机用户的信息安全意识,提高个人的信息安全水平,减少针对系统的非法入侵和攻击带来的损失。因此,进行密码分析方法的研究和新的攻击方法的挖掘,具有极其重要的现实意义。

1.2 国内外研究现状

在目前的密码学领域中,存在类型繁多的攻击方法。鉴于这些方法可能会对信息的安全造成威胁,因此只要方法面世,针对这种攻击手段的补救措施就会出现。虽然旧的攻击方法已经不能够利用,但可以在对其认真分析的基础上,找到新的攻击思路。一个系统不可能是绝对安全的,有时对已有漏洞的修复,甚至还会引入新的安全隐患。本文提出一种新的针对 AES 加密算法的攻击模型,主要涉及到缓存攻击和 Padding Oracle 漏洞这两种侧信道方法,因此下面分别介绍他们的发展现状。

1.2.1 缓存 (Cache) 攻击

密码算法在执行的过程中,对内存访问存在着数据依赖性,这种现象使得攻击者可以获得缓存的失效或命中信息。很早之前,计算机高速缓存背后的安全隐患就被提出了^[6],但是具体可行的攻击理论和方法的研究,直到最近才系统化。缓存攻击利用计算机高速缓存在工作时的表现的行为特征,如功耗的差异、时间的差异、缓存的命中与失效(主要)等,然后结合算法工作时候访问内存的方式,使用统计学的工具或方法,最后推断出一些敏感信息,如离散的密钥位。缓存攻击是一种典型的侧信道攻击方法,从信息论的角度来看,缓存的失效与命中泄露了算法的内部状态的信息熵,因此攻击得以实现。

根据利用的信息不同,缓存攻击可以分为指令缓存攻击和数据缓存攻击,两者有本质上的区别。前者主要利用了密码算法工作流程与密码算法在执行过程中指令调用顺序的关系,进行数据的分析与推导,如可以通过检测某一指令在缓存中的变化,推断出密码算法中某一函数是否被执行。数据缓存攻击利用了数据的访问方式进行攻击,如在 AES 加密算法中,每一轮都有 S 盒的置换,这种查表操作会有内存访问方式的变化,而这种变化与 AES 使用的密钥具有强烈的依赖关系,对这种关系进行分析后,密钥私密信息就会泄露。

缓存攻击可广泛应用到一切实现于“Cache-Memory”层次存储结构上软件形式的加密算法,涉及到服务器、桌面、嵌入式等各种主流计算机系统。主要发展如下: Kelsey 等^[8]在 1998 年率先提出了缓存命中和失效侧信道泄露差异可以用于密码分析的思想; 2002 年, Page 等^[9]在仿真缓存环境下首次进行了 DES 密钥分析, 他们利用 DES 在加密时 S 盒中数据索引和密钥的相关性, 结合此时缓存的命中与失效信息, 将破解 DES 密钥的空间复杂度大大降低: 由 2^{56} 降低到 2^{32} ; 2003 年 Tsunoo 等在真实环境下对 DES、AES、Camellia 进行了缓存计时攻击^[10-11]; 2004 年 Bernstein^[12]首次在远程环境下对 OpenSSL 中的 AES 算法进行了缓存计时攻击, 成功地从本地获取到了服务器上使用的 AES 密钥; 2005 年 Bertoni 等^[13]首次在仿真的环境下实现了 AES 密码缓存功耗分析, 最多可以得到 128 位 AES 密钥中的 80 位; Perciva^[14]等在 2005 年首次提出针对 RSA 密码算法的数据缓存攻击方法, 成功获取 512 比特的幂指数中的 200 比特, 同时还提供了可以恢复整个密钥的方法; Aciiçmez 等^[15]在 2007 年提出了针对 RSA 公钥密码的指令缓存计时分析, 并利用格攻击计算出了私钥; 2009 年 Ristenpart 等^[8]提出了基于云的缓存攻击, 他们首先使用称之为“云制图”的网络扫描策略, 得到了基于云的虚拟机(Virtual Machine, VM)位置, 然后放置一个恶意 VM 在相同的物理 Hypervisor 平台上, 随后访问共享硬件和缓存位置, 执行各种侧信道攻击, 包括拒绝服务(DOS)、硬件利用率检测、通过定时推理进行远程键入监控等; 2012 年, Zhang 等^[16]通过类似的基于云的缓存攻击方法, 从 GNU Privacy Guard 应用中得到了 4096 位的密钥信息; 2013 年 Yarom 等人^[17-19]使用一种称之为“Flush+Reload”的高精度缓存攻击手段, 对 RSA 和 ECC 进行了攻击, 可以分别恢复 96.7%的 RSA 密钥和 95.74%的 ECC 密钥, 且攻击能在很少的次数内完成; 2014 年, Zhang 等^[20]人使用缓存攻击实现了 PaaS(Platform-as-a-Service)平台上的交叉客户攻击(Cross-Tenant Attack), 可得到应用的敏感数据。

密码分析方法多种多样, 缓存攻击是近年来被提出的方法的一种, 目前对这种方法在理论研究及实际应用中仍然存在一些缺陷, 而已取得的相关研究成果也不尽完善, 仍有许多有待改进的地方, 并且在理论及形式化的研究中还未形成完善的架构体系, 影响了这种创新型密码分析方法对实际工作的应用。因此。创新型攻击及防御研究在理论及应用的广阔的前景应该被重视, 开展相关研究工作是十分必要的。

1.2.2 Padding Oracle 漏洞

Padding Oracle 漏洞^[21]是由 Vaudenay 于 2002 年首先提出的, 攻击手段针对的是对称加密算法中的 CBC (Cipher Block Chaining) 模式。CBC 模式是一种分组链接模式, 在分组密码中有着最广泛的应用, 它的目的是为了使原本独立的分组密码加密过程形成迭代, 使每次加密的结果影响到下一次加密。使用 CBC 模式进行加密的明文, 其总的长度必须满足是分组长度的倍数, 因此, 大多数情况下, 需要对明文进行填充。密文在解密完成后, 算法会检查这个填充是否合法, 如果不合法, 则会返回一个反馈信息 (Oracle), 攻击者如果可以获取这个信息, 则可以达到攻击的目的。

当加密算法使用 CBC 模式时,如果满足攻击条件,那么利用 Padding Oracle 漏洞能够在不知道密钥的情况下,解密任意密文。漏洞并不会提升攻击者的权限(如获取一个 shell),也不能执行恶意代码,但是该漏洞会泄露一些用户的私密信息,如在浏览器中使用的 cookie。这些加密后的信息的获得是不需要密钥参与的,被恶意窃取的私密信息可能会给系统带来严重的威胁。漏洞发布随后的几个月,OpenSSL 等开源库修复了此漏洞:加密 Oracle 信息或使 Oracle 不存在。2003 年 Canvel 和 Vaudenay 等^[22]利用了 TLS 中的消息认证码(MAC)再次获取 Oracle 信息,得到了基于 SSL/TLS 的 IMAP 账户密码;同年, Bardou 等^[23]验证了 Padding Oracle 漏洞在 USB 安全令牌、智能卡上同样具有危险性;2004 年 Paterson 等^[24]使用类似 Vaudenay 的方法证明了使用 ISO CBC 模式加密时,明文的比特可以恢复出来;2010 年 Rizzo 等^[25]详细介绍了 Padding Oracle 漏洞的实现步骤,利用此漏洞,在不知道密钥的情况下,可以解密任意密文或者构造出任意明文的合法密文,而无需知道密钥;2012 年 Bardou 等人^[26]利用 Heartbleed 信息重新构建了 Oracle 信道,完成了对 OpenSSL DTLS 的攻击。2013 年 AlFardan 等提出的“Lucky Thirteen”^[34]攻击,这种方法利用密码算法在处理合法填充和不合法填充时调用 HMAC 算法次数的不同引起的时间差异获得 Oracle,进而展开攻击。

目前国内关于 Padding Oracle 的研究很少,特别是在理论方面,国内的研究还很薄弱,在实践方面也只有少数几篇关于攻击原理的博客。但因 Padding Oracle 漏洞在实践过程中易于实现,且不需要直接得到密钥即可恢复明文,所以这种攻击和其衍生攻击手段会给网络通信造成较大的危害,对其进一步的研究具有非常重要的现实意义。

修复 Padding Oracle 漏洞的思路是很清晰的,那就是阻断获取 Oracle 的信道。如 Canvel 和 Vaudenay 的攻击思路是:在 OpenSSL 实现的 TLS 中,若解密得到的正确,则后面会执行消息认证码(MAC)的校验;若填充不正确,则直接中断会话。而 MAC 校验过程需要耗费一定的时间,这就意味着可以利用这个时间差,间接地得到 Oracle 信息。因此,开源密码库 OpenSSL/GnuTLS 等在解密密文的时候,会强制执行消息验证码的校验,无论填充正确与否,这就避免了时间测信道信息的泄露。因此,构建新的 Oracle 信道,获取解密过程中的 Padding Oracle,成为了重新利用 Padding Oracle 漏洞展开攻击的关键。

1.3 论文研究的内容

1.3.1 论文的研究思路

对称分组加密算法用于数据的私密传输,广泛用于当今的通信活动中,因此,对分组加密算法的安全性进行研究,具有很重要的现实意义。Padding Oracle 漏洞是一个危险性较强的安全隐患,它的目标是使用 CBC 模式的分组加密算法。目前已被提出的基于 Padding Oracle 漏洞的攻击方法,均被一一修复无法利用。新的 Padding Oracle 攻击手段的发现,对进一步保障信息的传输安全以及相关研究工作,有着举足轻重的作用。

重新利用 Padding Oracle 漏洞进行攻击的关键是构建可以获取有效 Oracle 的信道。鉴于之前的方法均已失效，本文将网络攻击和硬件攻击的方法结合起来，攻击思路如下：使用指令缓存攻击，获取加密算法运行时生成的错误信息，然后使用标准的 Padding Oracle 漏洞，得到解密后的部分或者全部密文。攻击手段的可行性在文中可以用密码算法的安全性来论证。

研究的内容有以下几个方面：

- 1，首先从分组加密算法（本文以 AES 算法为例）的安全性缺陷入手，分析本文提出的攻击模型是否会对密码方案造成威胁，并推导攻击模型的效率在数学上的表达。
- 2，设计出精确高效的指令缓存攻击模块，该模块可以达到监测加解密进程中，算法执行步骤的目的。通过监测算法的执行步骤，完成 Padding Oracle 的获取。
- 3，设计需要进行攻击的目标程序模块，使用开源库自行编写；最终使用攻击模块完成从 Oracle 信息获取到推断出明文的过程。

1.3.2 论文的主要贡献

本文基于对称加密算法中 Padding Oracle 漏洞，提出了一种新的缓存攻击模型，并通过实验验证该模型具有同样的攻击效果。攻击模型与之前的基于 Padding Oracle 的攻击方法相比，不需要构造复杂的数据包，获取 Oracle 的途径直接明了，因此在恢复明文方面具有高精度的特性。同时，本文也从加密算法 CCA 安全缺陷的角度，论证了攻击模型的有效性，同时给出了攻击效率的数学表达。

本文的创新点有如下几个方面：

- 1，在已有 Padding Oracle 获取方式被一一修复情形下，本文提出了一种新的攻击模型，模型引入了缓存攻击，构建了新的 Padding Oracle 获取信道，形成了新的攻击方法；
- 2，本文提出的攻击模型首次把实际的攻击手段与 CCA 安全结合，从数学上证明了攻击的合理性与有效性，同时对攻击效率及噪声优化有了定量的描述；
- 3，同已有的攻击手段相比，缓存攻击获取 Oracle 的方法更加直接有效。与最新的研究^[34]中的攻击成功率（双字节恢复为 80%）相比，使用本文方法展开的攻击具有更好的精度和成功率（99%）。
- 4，本文提出的攻击手段不仅适用于单台计算机，还可以迁移至云平台上。分布各处但共享同一硬件平台的多个云主机，满足本文提出的攻击方法的攻击条件。

1.4 论文的组织

本文基于加解密算法中的缓存攻击框架，提出了一种新的 Padding Oracle 攻击新方法，论文分为 7 章，其余章节的安排如下：

第二章详细介绍了 AES 加密算法和它的安全性。本文通过数学形式化定义了两种针对 AES 算法攻击实验，AES 在这两个实验中安全性表现不一样，其中弱的安全表现给后文提出针对 AES 算法的攻击模型提供了灵感。

第三章介绍了在密码算法安全领域的两种侧信道攻击：缓存攻击和 Padding Oracle 攻击，虽然这些攻击方法已经失效了，但是对它们的进一步研究可以为新的攻击形式的提出、新的攻击理论的建立提供必要的启发。

第四章提出了一个攻击模型，该模型建立在第二章的数学基础上，它的各个组成模块会在本章中一一详述。利用该攻击模型，本文提出了两种不同场景下的攻击实验设计。

第五章是攻击实验的实验过程及结果分析部分。

第六章首先总结了本文的研究工作，然后给出了若干个可以值得继续深入研究的方向，同时也指出了本文研究工作中存在的一些问题。

第二章 数学理论

高级加密标准 (AES, Advanced Encryption Standard) 2000 年由美国国家标准和技术协会 (NIST) 公布。AES 加密算法实现上能够兼顾软硬件平台, 并且能有效抵抗线性分析和差分分析, 因此它成为了当今世界使用最广泛的对称加密算法, 用来数据传输过程中的敏感信息, 保障通信的安全。

本章首先介绍了 AES 加密算法的原理, 包括工作流程和几种常见的工作模式。然后从现代密码学计算安全角度出发, 对 AES 这种加密方案进行了数学上形式化的定义, 基于此定义, 着重介绍了 AES 加密算法在选择明文攻击 (CPA) 选择密文攻击 (CCA) 和下的表现。本章是第四章所提出的攻击模型的理论基础。

2.1 AES 加密算法原理

2.1.1 AES 算法的工作流程

1997 年 NIST 宣布会举办一个比赛来选出一个新的分组加密算法来代替 DES, 这个比赛收集到了来自全世界各地的 15 种算法, 最终获胜的算法是 Rijndael (由来自比利时的 Vincent Rijmen 和 John Daemen 设计), 即 AES 的前身。从技术来讲, Rijndael 和 AES 是不同的, 但是其区别对下面的讨论并不重要, 以下统一以 AES 代替论述。

AES 算法属于分组加密, 其分块长度为 128 比特, 密钥长度有三个版本, 分别是 128 比特、192 比特和 256 比特三种。不同的密钥长度会对子密钥的个数和加密轮数产生影响, 其中, 使用 128 比特的加密轮次为 10 轮, 192 比特密钥对应 12 轮, 256 比特密钥有 14 轮, 需要注意的是, 密钥长度对加密机制中的高级结构并不产生影响。从本质上来讲, AES 算法的结构是不同于 DES 算法的, 前者的结构是一个代替置换网络, 而后者为 Feistel 结构。

在 AES 的计算中, 一个字节可以看成有限域上的一个元素。加密和解密中每一步产生的 128 比特的结果, 称之为一个状态 (State), 它是一个 4 乘 4 的字节数组, 其初始状态等于分组密码的输入, 每轮操作都会进行修改。下面是每轮加密中的四个基本步骤, 如下图 2.1 所示:

1. 字节代替 (SubBytes)。这一步是将状态中的每个字节通过一个替换表 (称为 S 盒) 换为一个新的字节, 从而进行非线性混淆。替换的实质是由)上的乘法逆运算和上的仿射变换所组成。
2. 行位移变换 (ShiftRows)。将状态中的每一行分别循环移动 0、1、2、3 个位置, 因此该操作是完全线性的。
3. 列混合 (MixColumns)。将状态中的每一列进行可逆的线性变换。具体做法是

每一列乘上一个 4 乘 4 的矩阵。

4. 子密钥变换 (AddRoundKey)。这一步操作的作用是把轮密钥加到状态中去, 即状态密钥和轮密钥进行有限域上的加法 (整数域上的异或)。

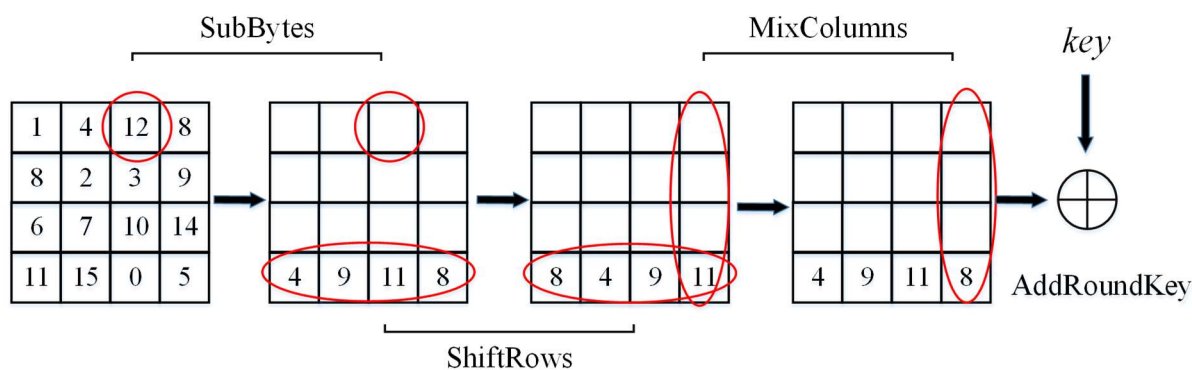


图 2.1 AES 算法单轮加密状态变化

整个 AES 加密算法由下面的步骤构成:

1. AddRoundKey (round = 0)
2. for round = 1 到 N (N 为轮数, 其值因密钥长度不同而不同)
 - SubBytes
 - ShiftRows
 - MixColumns
 - AddRoundKey (round)
3. SubBytes
4. ShiftRows
5. AddRoundKey ()

2.1.2 AES 算法的加密操作模式

操作模式是指用分组密码加密任意长度明文的方法, AES 有 5 种基本操作模式, 分别是电子密码本 (ECB)、密码分组链接模式 (CBC)、密码反馈模式 (CFB)、输出反馈模式 (OFB) 以及计数器模式 (CTR), 其中前四种模式也被 DES 加密算法所使用。其他衍生的算法操作模式, 都是建立在这五种模式基础上的。

实际中加密的明文长度很少是分组长度的倍数, 如果要使用某些模式 (如 CBC) 进行分组加密, 需要在加密前对明文进行填充, 填充的方案有多种^[27]。如在 DTLS 中规定, 若明文 m 缺少了 p 个字节, 则最后一个明文分组填充 p 个数值为 $p-1$ 大小的字节。以下各种模式中, 默认明文已经填充为分组的整数倍, 且定义明文分组和密文分组分别为 $m_0, m_1, m_2, \dots$ 和 $c_0, c_1, c_2, \dots$ 。

1. 电子密码本模式 (ECB)

这种模式直接对明文分组进行加密, 具有简单易行, 可并行化及误差不传递的优点。

但这种模式无法实现保密性，如下图 2.2 所示，相同的明文分组被多次加密后，就会泄露信息，这在相同的会话中以及不同的会话中都会出现。

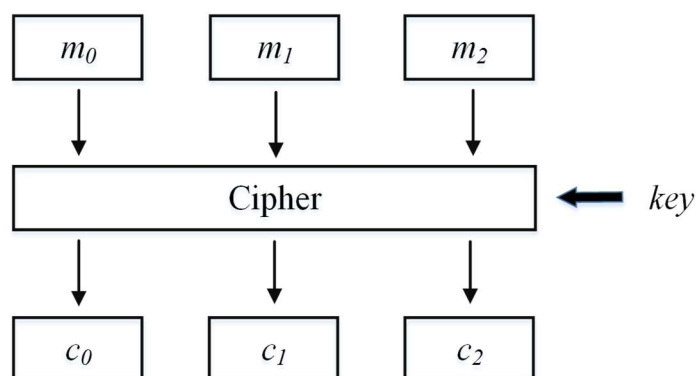


图 2.2 电子密码本链接模式 (ECB)

2. 密码分组链接模式 (CBC)

CBC 模式是 SSL/IPSec 的标准，广泛应用于对话中的加密传输，它克服了 ECB 模式安全性弱的缺点。在这种模式下，每个明文分组在加密前需要和一个初始向量 (IV) 做异或操作，然后做加密操作，得到密文分组。 IV 具有和密码分组同样的长度，它的选取必须随机。在会话中， IV 通常是以明文的形式和密文一起传送的。

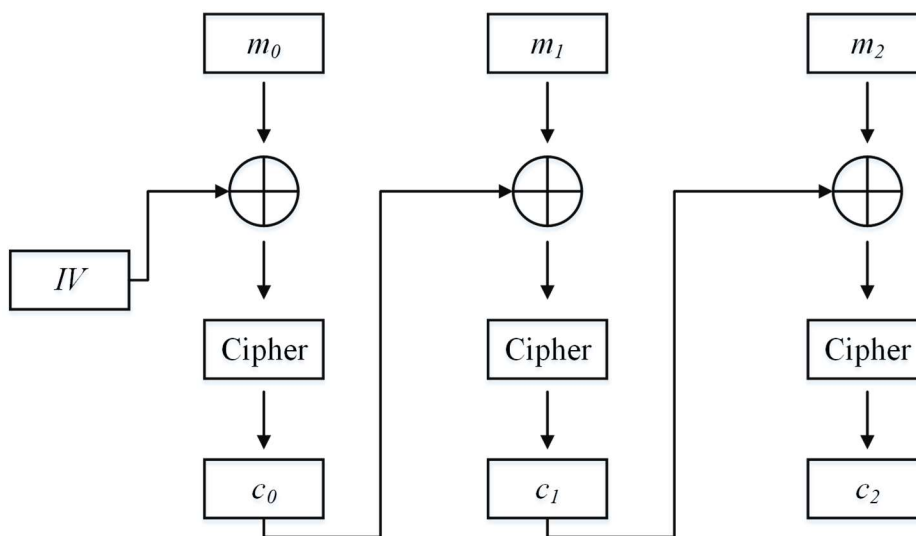


图 2.3 密码分组链接模式 (CBC)

上图是 CBC 模式的示意图，可以明显地看到，该模式在加密的时候会面临串行化的问题，密文只有在 c_n 只有在 c_{n-1} 被计算出来后才可算出，这使得密码不能以并行的方式进行调用。

使用 CBC 模式进行解密，只需要把密文传给解密机，并把解密机的输出和链接值进行异或即可。

3. 密码反馈模式 (CTR)

计数器模式被设计用来解决 CBC 模式中遇到的无法并行化的问题。在这种模式中，

可以像流密码一样对明文进行加密。如图 2.4 所示：

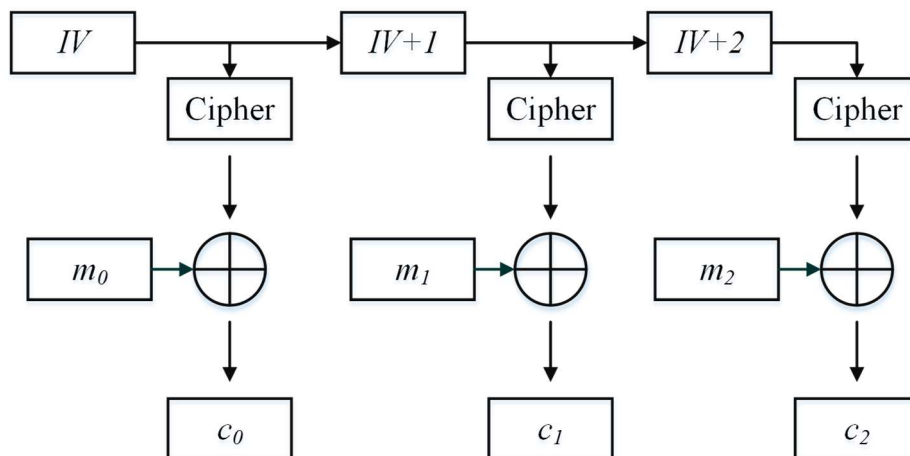


图 2.4 计数器链接模式 (CTR)

上图中有个类似于 CBC 模式中使用的 IV ，不同的是它并不要求是随机的，但需要保证其唯一性。另外的不同是，CTR 模式中是将 IV 进行加密，然后和明文分组进行异或， IV 的拓展是透过递增 (Increment) 来实现的。

在 CTR 模式中，其加密和解密的过程是一样的，这就意味着其实现更为简单。

4. 输出反馈模式 (OFB)

这种模式是使用分组密码来产生伪随机流，然后和明文进行异或操作。

图 2.6 中使用伪随机函数 F 表示伪随机流的产生过程，可以看出，这种模式有个优点就是加密明文可以独立于产生伪随机流的运算，通常后者需要的计算量很大。OFB 模式的解密操作和加密操作一样，即加密机和解密机为同一个。

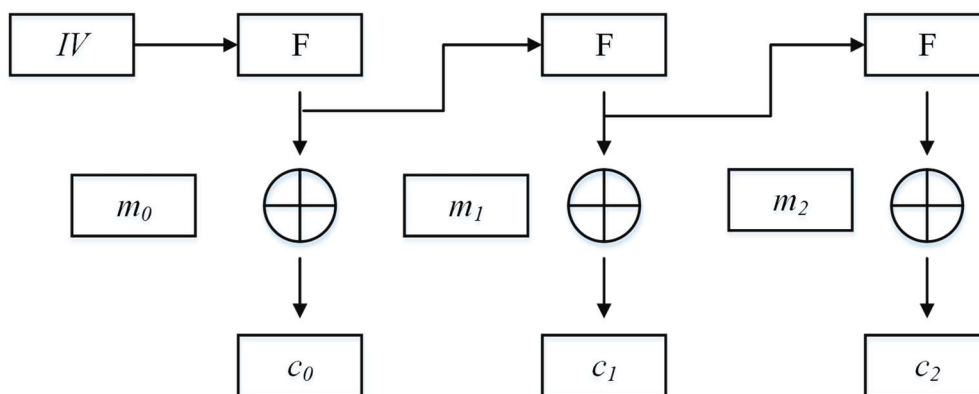


图 2.5 输出反馈链接模式 (OFB)

5. 计数器模式 (CFB)

同 OFB 模式一样，CFB 也是先产生一个密钥流，然后和明文进行异或，不同点在其产生伪随机流/密钥流的方式不同，加密过程见图 2.6。

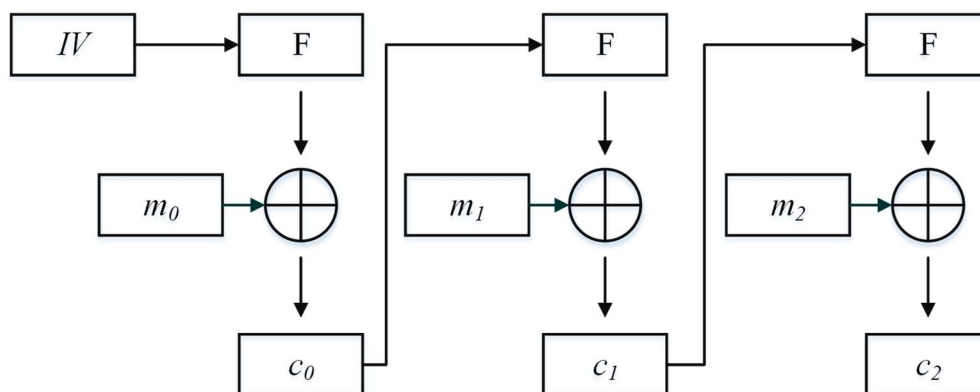


图 2.6 密码反馈链接模式 (CFB)

2.2 AES 算法的安全性

2.2.1 对称密码方案的安全性

在定义对称密码方案的安全性之前，需要明确理论安全和计算安全这两个概念。一个密码方案是理论安全的，意味着即便是攻击者有着无限的计算能力，也无法攻破此方案；计算安全是指，攻击者在可行的时间内（通常指的是多项式的时间），攻破密码方案的概率可以忽略不计。计算安全包含了两个放宽了的条件：一是攻击者的攻击时间是在可行的时间内，方案才是安全的；二是攻破的概率特别小，小到以至于不用考虑这件事是否会发生。本文后续章节分析中的“安全性”，仅指的是计算安全。这种安全分析方法在实践中是有效的，因为很少有人去考虑会一个持续几十年的攻击策略。

对现代密码学安全性精确的定义和证明，源于 Goldwasser 和 Micali 的开创性论文^[28]。这篇论文引入语义安全的概念，并论证了语义安全实质上等价于不可区分性。使用语义安全，可以系统且完整地表述一个密码系统的安全程度，但因为语义安全的范畴复杂且困难，通常使用不可区分性来证明一个密码学构造是否安全会更直接且有效。在正式介绍不可区分的定义之前，先使用数学语言重新定义对称密钥加密这种密码学方案。

规定 n 为对称加密中的安全参数，其值的大小可认为等于密钥的长度，如 AES 中， n 的取值可以是 128/192 或者 256。使用 $\{0, 1\}^n$ 来表示长度为 n 且由 0 和 1 组成的二进制字符串，使用 1^n 来表示长度为 n 的输入。

一个加密方案（如 RSA，AES）可以用 Π 来表示，同时 $\text{PrivK}_{A,\Pi}$ 表示攻击者为 A ，加密方案为 Π 的攻击结果，结果为 1 说明攻击成功，为 0 表示攻击失败。

定义 1: 对称加密方案由密钥生成算法 Gen ，加密算法 Enc ，解密算法 Dec 组成，可表示为三元组 $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ ，其中：

密钥生成算法 Gen 可以用来生成加密过程中所需要的密钥，其输入为 1^n ，输出为 k ，记作 $\text{Gen}(1^n) \rightarrow k$ 。

加密算法 Enc 将密钥 k 和明文 m 作为输入，输出密文 c ，记作 $\text{Enc}_k(m) \rightarrow c$ 。

解密算法 Dec 将密钥 k 和密文 c 作为输入, 输出明文 m , 记作 $Dec_k(c) \rightarrow m$ 。

需要注意的是, 上面表述中, 加密算法 Enc 中的参数及结果: k, m, c 和解密算法 Dec 中的 k, m, c 应该相等, 且三者的值均在 $\{0, 1\}^n$ 中。

上述方案 $\Pi = (Gen, Enc, Dec)$ 刻画了一个对称加密系统的内容, 而不可区分性则描述了该系统的安全属性, 可被简单叙述为: 加密两份明文中的一份, 随机猜测使得明文和密文相符的概率为 $1/2$, 如果一个攻击者不能以高于 $1/2$ 的概率去判断出给定密文对应的明文, 则该加密方案 Π 是安全。完整得到定义^[35]如下:

定义 2: 不可区分实验 $PrivK_{A,\Pi}$:

- 1, 给攻击者带有参数 n 的输入 1^n , 攻击者 A 根据该输入输出一对明文 m_0, m_1 ;
- 2, 使用密钥生成算法生成密钥 k , 然后从集合 $[1, 0]$ 中选取一个值 b' , 使用加密算法 Enc_k 加密明文得到密文 c , 然后将 c 交给攻击者 A ;
- 3, 攻击者拿到密文后, 使用自己的方法得到输出 b' ;
- 4, 如果 $b = b'$, 则可表示为 $PrivK_{A,\Pi} = 1$, 说明攻击成功, 反之 $PrivK_{A,\Pi} = 0$ 。

AES 属于对称加密方案, 因此可以在不同的攻击场景下, 基于不可区分性对 AES 在该场景下的安全性进行评估。如果攻击者在攻击实验中达到了可区分的目的, 则说明攻击成功, 也就意味着在该攻击实验下的密码方案存在攻击弱点。

在使用对称密钥进行加密的实验中, 根据攻击者 A 获取信息的不同, 考虑下面两种攻击实验: 选择明文攻击不可区分实验 (下文简称 CPA) 选择密文不可区分攻击实验 (CCA)。

2.2.2 选择明文攻击 (CPA)

在选择明文攻击 CPA 中, 允许攻击者 A 可以获取其选择明文对应的密文, 即攻击者 A 和加密方案有了一定的交互。在这里, 可以将加密算法视为“加密黑盒”, 攻击者 A 在不知道密钥 k 的情况下提供一串明文给“加密黑盒”, 然后得到加密之后的密文。考虑上节中定义的不可区分实验, 可得到 CPA 不可区分实验的过程为:

定义 3: CPA 不可区分攻击实验 $PrivK_{A,\Pi}^{CPA}(n)$ ^[35]:

1. 通过运行 $Gen(1^n)$ 生成一个密钥 k ;
2. 攻击者 A 被指定输入 1^n , 可以使用 Enc_k 进行伪随机变换, 输出一对长度相等的消息 m_0, m_1 , 消息对攻击者而言不是私密的;
3. 从集合 $[1, 0]$ 中选取一个值 b' , 计算密文 $c = Enc_k(m_{b'})$ 的值, 然后将其提供给 A 。把 c 叫做挑战密文。
4. 攻击者 A 继续使用 Enc_k 。最终 A 输出一个比特 b' 。
5. 如果 $b = b'$, 实验的输出为 1, 意味着攻击成功, 否则输出为 0。

上述实验中的攻击者 A 已经具有了相当强大的攻击能力,可以自由选择明文进行加密。可以看出,如果上述的加密方案是确定性的(即每次同一个明文加密可以获得同样的结果),则上述的攻击者可以达到攻击目的:他可以将 m_0 和 m_1 都进行加密得到两份密文,然后和挑战密文进行对比,即可判断出 b 的值。因此,对于确定性的加密方案,如使用 ECB 模式的 AES 加密,就存在这个问题,该模式下的各个分组独立,攻击者对调任意分组的明文,得到加密过后的密文也是对调的。因此,对一定明文产生一定密文的确定性的加密方案都是不安全的。

如果要保证 CPA 安全,显而易见要在加密方案中引入随机性,使得对同一个明文进行不同的加密,产生的结果是不同的(存在概率性)。CBC/CTR/OFB/CFB 模式下的 AES 加密拥有这个优点,初始化向量 IV 的引入,带来了概率性的加密结果。事实上,上述四种模式中的加密算法/解密算法(均视为“黑盒”)在数学上是被当作伪随机函数来进行分析,它将一个输入明文均匀地输出,得到一个伪随机的密文。因此, CBC/CTR/OFB/CFB 模式下的 AES 加密方案,具有抵抗 CPA 攻击的特性。

2.2.3 选择密文攻击(CCA)

考虑将攻击者 A 的能力继续提高,他不仅能和加密算法 Enc 进行交互,还可以访问解密算法 Dec ,这种攻击称之为 CCA 攻击实验。定义如下:

定义 4: CCA 不可区分攻击实验 $\text{PrivK}_{A,\Pi}^{CCA}(n)$ [35]:

1. 通过运行 $\text{Gen}(1^n)$ 生成一个密钥 k ;
2. 攻击者 A 被指定输入 1^n , 可使用 Enc_k 和 Dec_k 进行伪随机变换, 输出一对长度相等的消息 m_0 、 m_1 , 消息对攻击者而言不是私密的;
3. 从集合 $[1, 0]$ 中选取一个值 b' , 计算密文 $c = Enc_k(m_{b'})$ 的值, 然后将其提供给 A 。把 c 叫做挑战密文。
4. 攻击者 A 使用 Enc_k 和 Dec_k , 最终 A 输出一个比特 b' 。
5. 如果 $b = b'$, 实验的输出为 1, 意味着攻击成功, 否则输出为 0。

CCA 和 CPA 的过程很相似,唯一不同的就是: CCA 除了可以自由选择需要加密的明文,还可以使用解密算法对选择输入的密文进行解密,得到解密的结果。注意上面定义中的第 8 步,对攻击者的限制是不能直接解密挑战密文,否则整个实验就没有意义,没有任何密码方案可以抵抗这种攻击实验。

AES 的五种加密模式中,已经被证明均不是 CCA 安全的[29]。其根源是具有 CCA 安全的密码方案,必须具有“不可延展性”,即如果攻击者提供给 Dec_k 一个修改过的密文, Dec 必须做出以下两种反馈中的一种: 1, 提示“非法密文”; 2 输出的结果和明文没有任何关系,但上述的五种操作模式,都不具备这个特点,很多针对 AES 的攻击手段,都是基于 AES 在这些模式下存在 CCA 安全缺陷展开的。

上述对称密钥加密的各种分组链接模式之所以还被广泛使用,是因为现在的加密策略是将 CPA 加密方案和消息验证码 (MAC, Message Authentication Code) 结合在了一起。MAC 算法可以接受任意长度的输入,并输出一个固定大小的字符串。它的目标是保证消息传送的双方在通信的过程中具有检测消息是否被修改的能力:MAC 算法接受一个共享的私密密钥和消息,产生一个 MAC 标记,该标记会同消息一起被传送给接受消息的其他成员,然后可以重新计算标记,并把计算的结果和传送过来的 MAC 标记进行比对,如果相等,说明消息在通信过程中未被修改,否则消息应该被丢弃。这个过程可以有效地阻止 CCA 中攻击者对明文的修改,即攻击者的能力被削弱了,从而使得整个加密方案的安全性得到提升。

MAC 算法在实际使用过程中,有使用的先后顺序:先 MAC 再加密明文;先加密明文再 MAC。两种不同的过程如图 2.7 所示。第一种顺序 (2.7.a) 中,明文发送者在发送之前,先使用 CPA 安全的加密方案加密明文,然后对得到的密文计算出 MAC 标记,这两者一起构成了最后传送的密文。这种结合已经在数学上被证实是 CCA 安全的^[30]。第二种顺序 (2.7.b) 中,明文在发送之前,会先输入 MAC 算法得到 MAC 标记,然后把结果和明文一起进行加密,最后得到的密文是一个整体。这种使用 MAC 算法的组合,已经在理论上被证明是不是 CCA 安全的^[30]。

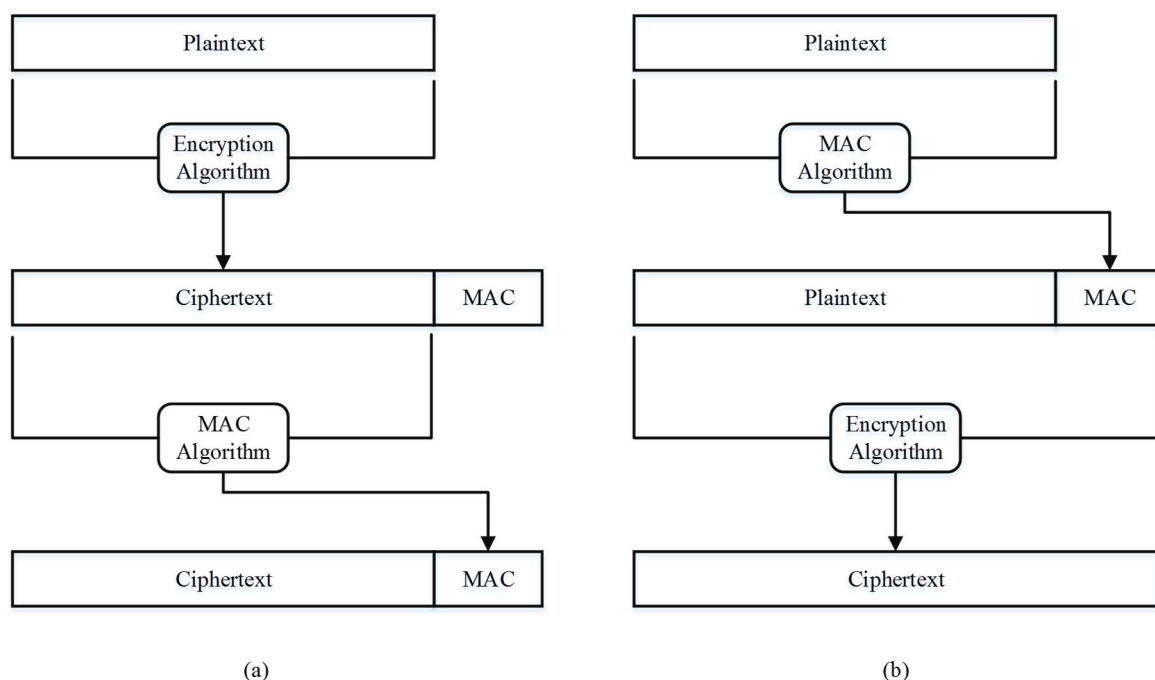


图 2.7 MAC 算法两种不同的使用顺序

然后在现有的 DTLS/TLS 协议的实现中,消息在传送的时候,使用的是先计算 MAC 标记,再进行加密这种策略 (图 2.7.b)。这是因为消息接收者在收到使用这种组合加密后的密文后,会不得不先进行解密操作,然后得到原始的 MAC 标记,最后验证 MAC 标记的有效性。这个过程在某些时候是一个缺点,因为每次无条件地进行解密,会造成一

定程度上资源的浪费，但同时它也引入了一个令人惊奇的优点：不会泄露时间信息给攻击者。考虑使用图 2.7.a 中的顺序加密后的密文，接收者收到后，会首先验证 MAC 标记，一旦失败了就可以终止处理，这就引入了时间侧信道的安全隐患，因为攻击者可以很轻易地得到什么样的伪造密文会造成处理失败的信息。

MAC 算法的使用本来是为了使得加密方案达到 CCA 安全，但是在实际的使用过程中，各大密码安全开源厂商为了避免不出现基于时间的信息泄露，采取了一种不安全的 MAC 算法和加密算法的使用顺序。这种策略在一定程度上消除了一类侧信道的攻击，但与此同时，这也使得 MAC 的作用减弱，降低了加密方案的 CCA 安全性，即实践安全性的提升导致了理论安全性的下降。

2.3 本章小结

AES 加密算法是现代通信中使用最广泛的对称加密算法，它具有高安全强度和使用简便的优点。同时，AES 也是密码安全领域的一个研究热点，研究者对密码方案本身，和它拥有的几种加密链接模式，进行了不断的探索。到目前为止，对 AES 算法理论上的攻击，还没有出现比穷举更高效的方法。因此，现有的攻击都是从 AES 在实践中的使用入手，结合侧信道的方式展开攻击。本章介绍了 AES 加密算法常用的几种分组链接模式，不同的模式都有其优点，也有不同的应用场合。在这几种模式中，CBC 模式是目前使用最多的。

如何在数学上对密码算法的安全性进行准确定义，一直是密码学研究重点。不可区分性是一个衡量密码方案安全的工具，它通过测试攻击者是否能区分密文所对应的明文，达到判断密码方案是否安全这一目的。根据攻击者的能力不同，针对对称加密密码方案的攻击，目前分为 CPA 和 CCA 两种，前者已经被证明是不足以对 AES 造成威胁的，而后者这种攻击类型则给 AES 引入了一个安全缺陷。

本章的内容为后文提出的攻击模型提供了理论支撑，攻击模型的提出基于 AES-CBC 模式存在的固有的 CCA 安全缺陷。攻击模型的实质还是加密方案的 CCA 安全性在实践中被破坏了，违背了安全加密的原则。

第三章 攻击基础

缓存攻击是一种重要的侧信道攻击手段，它主要利用密码算法在执行过程中通过缓存泄露的信息，结合密码算法本身的数学原理，展开对密码系统的攻击，是一种高效的攻击方法。本章首先介绍了缓存攻击的理论背景和发展概况，然后详细叙述了一种可以监测算法执行的缓存攻击方法。

本章第二节介绍了 DTLS 协议，这是一种传输层协议，用来保证数据的通信安全，该协议也是本文所提出的攻击模型的攻击目标。

本章最后着重介绍了 Padding Oracle 这种影响较大的攻击，它的攻击目标是 SSL/TLS/DTLS 协议中的对称加密算法；同时，这种攻击手段不以获取密钥为最高目标，而是通过分组链接中的缺陷，直接得到解密后的明文。

3.1 缓存攻击原理

3.1.1 缓存攻击基础

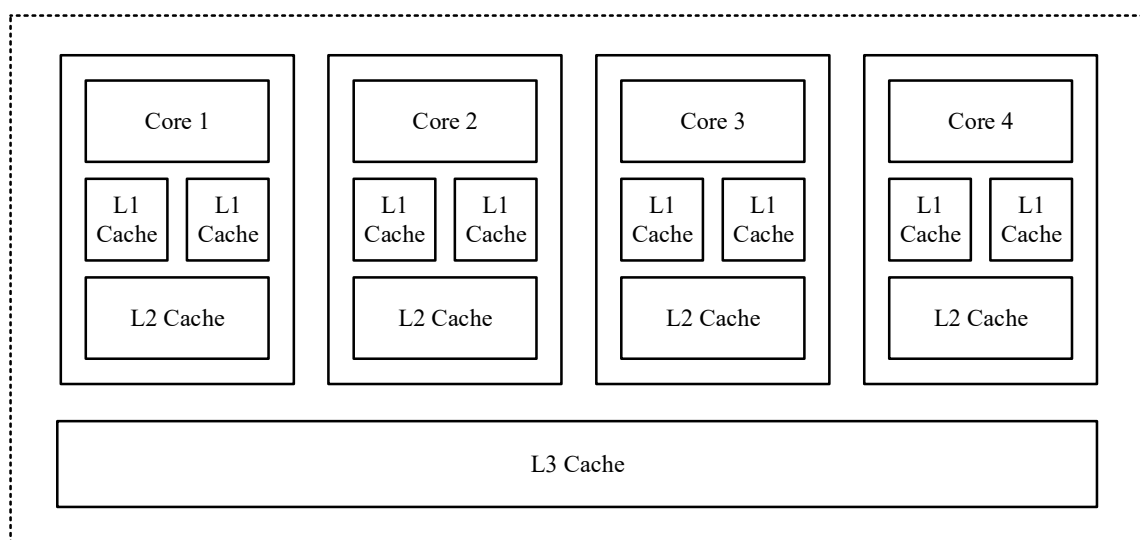


图 3.1 缓存的层次结构

早期计算机系统的存储器层次只有三层：CPU，主存和磁盘存储器。不过，由于 CPU 和主存之间逐渐增大的读取速度差距，系统设计者被迫在 CPU 和主存之间插入一个小的 SRAM 高速缓存存储器（Cache），称为 L1 Cache（一级缓存），它分为数据 Cache 和指令 Cache。L1 Cache 的访问速度几乎和 CPU 中的寄存器一样快，典型的是 2~4 个时钟周期。随着 CPU 和主存性能差距不断增大，又出现了 L2 Cache（二级缓存），可以大约在 10 个时钟周期内访问它。现代的微处理器还包括一个更大的高速缓存，称为 L3 Cache（三级缓存）。如图 3.1 所示的 Intel Ivy 处理器就是这种情况，它的 CPU 有四个

核，使用了三级缓存。Core 访问 L3 Cache 时的速度最慢，获取数据需要 30~40 个时钟周期。CPU 与缓存之间的数据交换是以字为单位，缓存与主存之间的数据交换是以字块为单位，最小的交换字块被称为 Cache line（高速缓存行），每一行由若干个字节组成。

缓存中存放着 CPU 最常使用的数据和指令。当 CPU 读取数据时，它首先会向缓存请求这个数据，如果缓存中有该数据的拷贝，那么就会出现缓存命中（Cache Hit），缓存会取出这个拷贝，并将它返回给 CPU；相反，若缓存没有该数据的拷贝，则会出现缓存失效（Cache Miss），缓存需要向主存请求，CPU 必须等待；若缓存中没有空间防止新的数据了，则需要算法将缓存中现有的 Cache line 替换出去，并对缓存和内存的映射关系进行修改。

CPU 执行密码算法程序的时候，就会发生缓存命中或失效的情况，这种行为可以通过各种侧信道的方式被泄露。如表现在功率消耗上，缓存命中时的功耗将显著小于失效时的功耗；表现在电磁辐射上，缓存命中与失效时的辐射信号将显著不同；表现在访问时间上，缓存命中的运行时间（约 30 个时钟周期）将显著小于失效的时间（近 300 个时钟周期）。缓存的这些侧信道信息是攻击者进行缓存攻击的基础，利用这些信息，结合密码算法的执行情况，攻击者就可以推断出一些敏感信息。

缓存攻击是一种非常有效的侧信道攻击手段，它的出现意味着针对通用计算机密码系统的攻击成为可能，对这种攻击手段的研究目前也越来越多。根据缓存攻击应用场合的不同，它可以分为以下三类：1）基于时间泄露的缓存攻击主要针对微处理器、嵌入式微处理器中的密码算法实现，时间信息可以在本地甚至远程环境下采集，在网络环境下也有望实现密钥的恢复。2）基于功耗、电磁的泄露的缓存攻击主要针对嵌入式微处理器中的密码实现，可利用示波器、电磁接收机等设备采集密码算法的执行功耗、电磁和时间二维信息，推断出密码算法在缓存上的访问命中和失效序列。3）缓存攻击除了上面的应用场合，还出现在云计算的舞台上。加利福尼亚大学和 MIT 的研究员利用缓存攻击揭露了基于云的 VM 的映射位置，通过称之为云制图的网络扫描策略实现^[8]，随后，研究者利用类似的方法得到了 GNU Privacy Guard 应用中的 4096 位的密钥信息^[16]。所以，云平台也面临着缓存攻击的潜在威胁^[31]。

3.1.3 缓存攻击实例

本文使用缓存攻击监测加解密算法的执行流，其原理是：由于最高一层的缓存（如图 3.1 中的 L3 Cache）是被所有进程所共享的，考虑到算法进程在执行时会使得缓存出现数据依赖性的若干特征，因此其他的进程如果获取到了这些特征，就可以其进行分析，分析的过程结合了算法的执行流和访问内存的关系，最后可以得到私密信息。攻击方法在实现的过程中，主要利用页面共享机制（“shared-page”），即一个进程的可执行文件可以映射到其他进程空间中，进程在工作的时候，两个进程空间对应部分对应磁盘上相同的文件，并且文件中的数据或者指令在缓存中的失效，会同时给两个进程带来影响。因此，通过同时运行攻击进程和加解密进程，攻击进程可以持续性地执行私有指令或者访问私

有数据,造成缓存中资源的冲突,从而达到监测加解密进程执行流的目的。此外,处理器的多个进程同时运行的能力为攻击的实现提供了可能,同时文献^[10]指出,也可以利用操作系统的调度机制,使得两个进程的轮流执行。

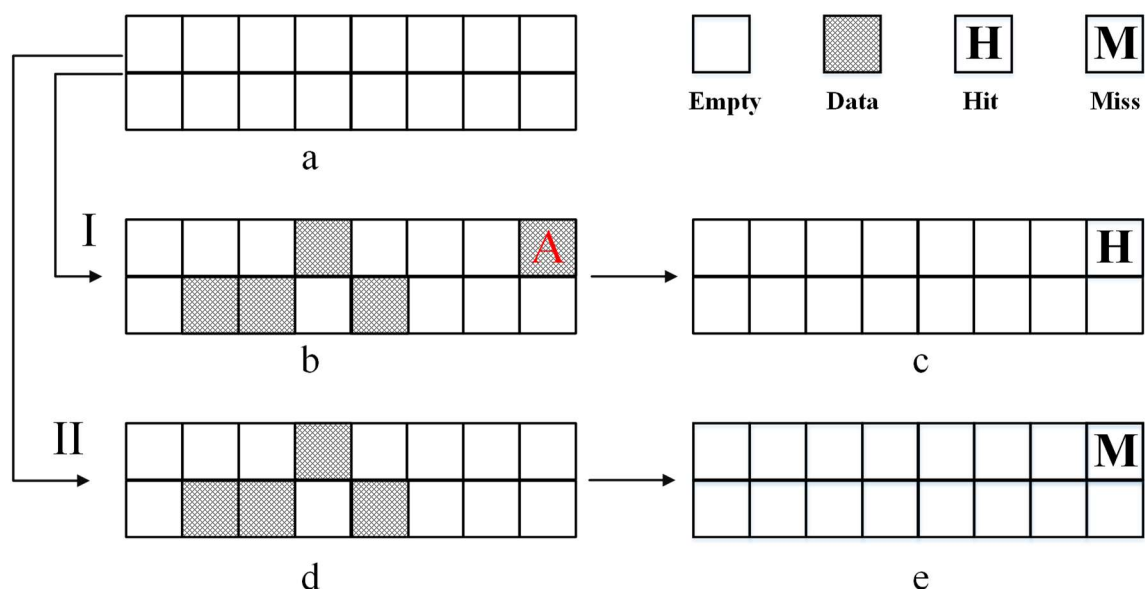


图 3.2 缓存攻击检测算法执行流

图 3.2 是使用缓存攻击监视算法执行流的原理,其过程可简要描述为:通过运行一个与密码进程(VICTIM 进程)同步的间谍进程(SPY 进程)监测特定指令(如右上角的指令 A)的存取情况,从而获取密码进程的执行流。图中每个小方块代表一个 cache line,它是内存和缓存之间交换数据或指令的最小单位。

在展开缓存攻击之前,需要将 SPY 进程映射到同 VICTIM 进程相同的地址空间,并允许 SPY 进程在 VICTIM 进程启动前能够执行代码且清空整个缓存。上述技术的细节,已经在文献^[14]中体现,文中对 AES 算法的最后一轮进行了攻击。文献^[32]提出了一种相似的攻击方法,攻击者可以通过改变 CPU 的周期,对 SPY 进程访问 VICTIM 进程的时间序列进行控制。预处理过后,就可以展开正式攻击了,每轮攻击分为三步:

- 步骤 1: SPY 进程清空指令 A 所对应的 cache line,如图 3.2 中(a)所示。
- 步骤 2: SPY 进程等待适当的时钟周期,让 VICTIM 进程执行加密等操作。若 VICTIM 进程在执行过程中访问了指令 A,则 CPU 会将指令 A 从内从中加载到缓存中,如图 3.2 中(b)所示;若 VICTIM 进程在执行过程中没有访问指令 A,则 CPU 不会将指令 A 从内从中加载到缓存中,如图 3.2 中(d)所示。
- 步骤 3: SPY 进程再次读取指令 A,计算读取指令需的时间,若小于时间阈值,说明刚才 VICTIM 进程已经读取过该指令了,会出现缓存命中,如图 3.2 中(c)所示;若时间大于设定的时间阈值,则说明 VICTIM 进程没有读取过该指令,会产生缓存失效,如图 3.2 中(e)所示。读取后会清空指令 A 所对应的 cache line。

SPY 进程以一定的频率循环执行步骤 1~步骤 3 的过程，直到 VICTIM 进程执行结束，则可采集到 VICTIM 进程的执行踪迹，即对应于算法的执行流。

在上述攻击的三个步骤中，SPY 进程都有读取指令 A 并驱逐(evicting)其所在 cache line 的行为，其关键的算法在 Yarom 等人^[21]的文献中有所体现，因为 CPU 读取数据/指令的时间很短，因此使用汇编实现。

表 3.1 中代码的工作原理是基于此事实：若 SPY 进程和 VICTIM 进程映射到了同一地址空间，则两个进程同时享有同一份共享的文件，若将其中一个进程的指令从缓存中驱逐，则它会直接体现在另一个进程相应的指令也被驱逐。这个过程可以使用 *clflush* 指令实现，该指令是 Intel x86 指令集中的内容。其功能是：*clflush address* 可以将线性地址/虚拟地址为 *address* 的数据在缓存中的内容清除掉，同时解除相应的内容-缓存映射关系。这种 Cache line 的失效会被应用在所有层级别的缓存中，即数据被驱逐后，在 L1、L2 和 L3 中，均不会存在内存到缓存的映射，下一次 CPU 如果使用该数据，需要重新建立新的映射关系。

表格 3.1 缓存攻击的算法描述

```

int probe_cacheline(char *addr)
{
    volatile u32 temp_time;
    asm __volatile__(
        "rdtsc"                //获取当前时间戳
        "movl %%ex, %%esi"      //保存当前时间戳
        "movl (%1), %%eax"      //读取 addr 地址对应的指令
        "rdtsc"                //再次读取时间戳
        "subl %%esi, %%eax"      //将两次时间戳相减
        "clflush 0(%1)"         //从缓存中将刚才读取的指令驱逐 (evicting)
        : "=a"(temp_time)       //temp_time 即为读取指令所用的时间
        : "c"(addr)
        : "%esi", "edx"
    );
    return temp_time < threshold_time; //将所用时间与阈值进行对比
}

```

代码中指针变量 *addrs* 代表需要监测的指令的逻辑地址，该地址的获取可通过反向工程得到^[33]。代码中的 *rdtsc* 可获取当前的时间戳，在进行指令读取的前后使用后，其差值可视为读取该指令的时间，在代码中用 *temp_time* 表示。将 *temp_time* 与提前设置好的时间阈值 *threshold_time*（可在前期用统计学方法得出）进行比较，若小于阈值，则说明监测的指令已被 VICTIM 访问过，若大于阈值，说明还未被 VICTIM 进程访问过。

使用上面的方法，通过监测单个或多个指令的执行情况，可以推导出密码系统的执行流，再利用执行流与算法的依赖关系（如某些函数的调用与密钥比特的取值相关）就能得到攻击者所需要的敏感信息。

3.2 数据报传输层安全协议

3.2.1 DTLS 协议概述

DTLS (Datagram Transport Layer Security) 是位于 TCP/IP 协议栈中应用层和传输层之间的一个协议，它建立在 TLS (Transport Layer Security) 协议的基础上，设计目的是为了解决使用 UDP (User Datagram Protocol) 传输信息时存在的安全隐患。由于 UDP 包与 TCP 包不同，后者有着完善的包传送控制机制，具备高可靠性。但是 UDP 协议在传送数据包的时候，仅仅只根据包中的 IP 地址进行发送，而数据包是否能被接收端收到，UDP 协议并不能保证这一点。同时，UDP 包的数据安全性，在 DTLS 协议提出来之前也未能得到保障。

DTLS 协议由握手协议、改变密码规格协议、警告协议、应用层数据协议和记录层协议组成，其中最重要是握手协议和记录层协议。握手协议负责通信双方在正式通信之前协商安全参数，如使用的加密算法类型、算法版本、HASH 算法等，记录层协议负责对通信信道中的消息进行加密鉴权。DTLS 协议的连接分为两步：第一步是握手阶段，进行身份验证和参数协商；第二步是数据传输阶段，这一阶段主要使用第一步中的密钥和算法进行信息的安全传输。

DTLS 协议是基于 TLS 协议设计的，它拥有非常好的安全性，同时也具有若干独特的优点。如 DTLS 协议进行数据传输的速度更快，DTLS 协议的消息重传功能保证消息在传输过程中不丢失，DTLS 协议的无状态 cookie 技术可以防止 DOS 攻击等。因此，DTLS 协议在信息安全越发被重视的今天，使用范围越来越广泛，如 VPN 和安全视频会议。

3.2.2 DTLS 会话安全

DTLS 协议的记录层对数据进行接受、处理和发送。数据在传输之前，会使用在握手协议中协商好的对称加密算法和密钥进行加密，整个传输的过程中，数据是以密文的形式存在的。被发送的数据并不是简单地被加密处理，在 DTLS 协议中，记录层采用一种“hmac-pad-encryption”的加密策略保证数据的机密性和完整性，防止数据在传输过程中被窃听或篡改。具体的工作流程见图 3.3。

图 3.3 中，待传输的消息（明文）被称为“Payload”，记录层首先在 payload 前添加上该数据特有的一些数据，如序列号 SN。填充好的 payload 会通过一个 HMAC 算法，得到一个 MAC（消息认证码），这个操作的目的是为了保证通信的双方具有检测传输消息被修改的能力，用来阻止攻击者篡改消息。HMAC 算法接受一个消息和一个密钥（在

握手阶段确定), 产生一个固定大小的 MAC 标记。

HMAC 算法执行完成后, 其结果被添加在消息结尾, 这个时候的消息就可以加密了。加密使用的对称加密算法, 一般是需要进行分组处理的, 因此如果消息不是分组长度的整数倍, 就需要进行填充操作, 填充的字节片段被称为 **Padding**。实现填充的策略有很多, DTLS 协议中使用的方法可简要描述为: 明文的长度如果需要 p 个字节补充才是分组的整数倍, 则在明文的最后依次填充 p 个数值为 $p-1$ 大小的字节。因此, 合法的填充会是像是“0x00”、“0x01, 0x01”、“0x02, 0x02, 0x02”……凡是不满足这种填充规则的填充, 密文在解密后算法会视为非法填充, 并反馈一个错误, 这个错误被称为 **Padding Oracle**。填充完成后的消息, 就可以使用协商好的对称加密算法进行加密了。加密操作结束后, DTLS 会在密文前添加 DTLS 包头, 接着, 这个 DTLS 包就被发送给通信的另一方了。接收者收到这个包后, 执行类似的操作: 解密密文, 检查填充, 校验 MAC。

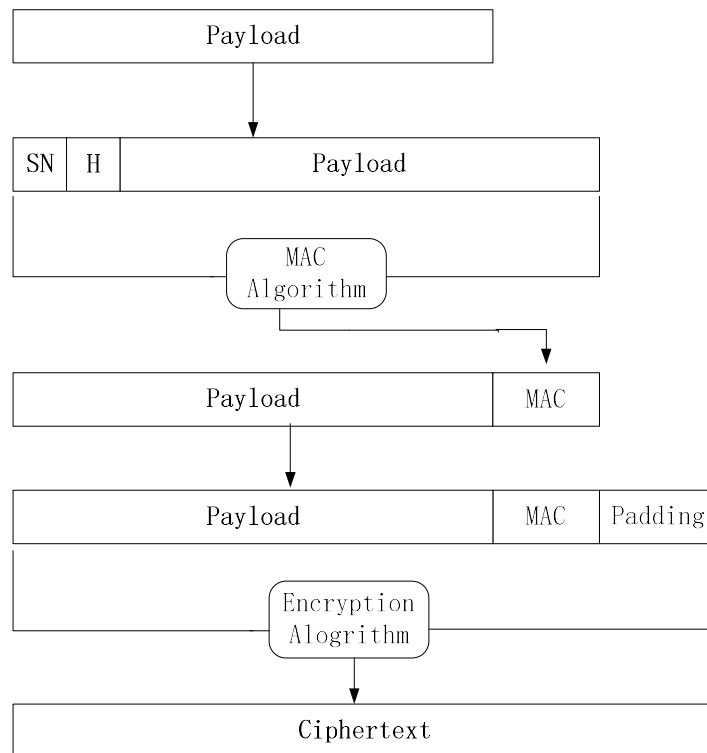


图 3.3 DTLS 协议中的加解密策略

3.2.3 常见的开源库实现

DTLS 协议用来保障 UDP 传输的安全, 它基于 TLS 协议设计, 而 TLS 协议又脱胎于 SSL3.0 协议。因此, 很多之前支持 SSL 的开源库都针对 DTLS 协议进行了开发, 其中最著名的两个为 OpenSSL 和遵循 GNU General Public License 发行的 GnuTLS。

GnuTLS 是一个自由的 SSL 和 TLS 协议的函数库, 它基于 ANSIC 函数库, 提供了一组用于使用安全连接协议的 API。这些安全协议能够在不安全的信道上实现安全通信, 并且具有防窃听、防篡改、防伪造的功能。GnuTLS 最开始的版本就实现了 TLS1.1 和

SSL3.0 协议以及这两个协议所必须的身份验证框架和公钥密码基础结构，不久之后，在 TLS 协议的基础上，开发小组又开发了 DTLS 协议，并且一直在不断地完善。

OpenSSL 是一个强大的 SSL (Secure Sockets Layer, 安全套接字层) 开源库，它提供基础的密码算法，如 AES/DES/ECC/RSA 等进行数据加密。同时 OpenSSL 还具备完善的证书管理及认证功能，可以提供通信时的密钥协商及身份确认。它支持的协议也很多，如 SSL3.0、TLS1.1、TLS1.2 等。从 0.98 版本开始，OpenSSL 库开始支持 DTLS 协议，到目前为止，DTLS 协议已经成为了 OpenSSL 开源库中的最重要的模块之一。

3.3 Padding Oracle 漏洞

3.3.1 漏洞原理概述

使用 CBC 模式的加密算法对明文进行加密，需要明文的长度是分块长度的倍数，如果不满足（绝大多数情况），则需要安全一定的规则在明文的最后进行填充，填充的内容即就是 Padding。相反地，密文被解密后，算法也首先校验 Padding 的有效性，检测其是否满足之前使用的填充规则，如果不符合的话，密码算法会反馈一个错误，这个错误就被称为 *Oracle*，它的取值有两种：合法填充，不合法填充。攻击者利用密码算法的这个加解密流程，可以进行如下攻击：

- 1, 攻击者伪造解密过程中需要使用的初始化向量 *IV*;
- 2, 攻击者将 *IV* 和密文一起发送给密码算法进行解密;
- 3, 密码算法揭秘后反馈给攻击者 *Oracle*;
- 4, 攻击者获得 *Oracle* 后，如果提示是不合法的填充，则回到第 2 步，否则继续;
- 5, 攻击者食用此时 *IV*，结合密码算法的数学运算过程，得到解密过程中的中间值，最后通过简单的异或操作得到密文对应的明文。

在 2010 年前的 Padding Oracle 漏洞研究，主要集中在理论上和实验环境中的研究。在这期间，明文的填充规则逐渐完善，相应地攻击的手段也拓展了不同的填充标准上。一些安全协议，如 TLS/DTLS，因为在加解密数据包的过程中也使用了填充，因此也出现了安全隐患。该漏洞取得最大突破时在 2010 年，文献^[25]给出了给予 web 网站框架的攻击方法，可以简要概述为：攻击者可以通过 URL 请求要求服务器解密提交的篡改密文，然后观察是否有错误提示发生。这些错误信息即为 Padding Oracle，它们可能以 HTTP 状态码的形式直接显示在网站，如“500 - Internal Server Error”表示此时检查填充的字节出现了错误，而“200 - OK”则表示解密正确，即填充正确。这种攻击的效率很高，恢复一个字节的明文只需要提交 256 次，假设每次 HTTP 网站的响应时间为 5s，恢复一个字节只需要约 21 分钟。随着研究的深入，研究者发现其他的框架或语言也存在这种漏洞，如 Ruby on Rails、JSF、PHP 等。

Padding Oracle 信息在最开始的实践中，是非常容易的，因为攻击者可以直接获得 *Oracle* 信息，随着漏洞被依次修补，攻击手段失效，研究者的研究重点逐渐放到了如何

通过侧信道获取 Padding Oracle 上面。常用的可被利用的侧信道信息主要是基于时间的侧信道泄露，如构造满足一定条件的密文进行解密，服务器的响应时间会有所变化，这种变化对应了填充的正确还是错误。

3.3.2 最新的漏洞构造

在已被公布的基于 Padding Oracle 的众多攻击方法中，大多数都是基于时间差获取 Oracle 信息的，即在解密过程中，不同的函数调用和调用顺序将会导致程序耗费的时间不同，这就为攻击者提供了一个侧信道。目前最新的 Padding Oracle 的攻击方法 2013 年 AlFardan 等提出的“Lucky Thirteen”^[34]，这种方法利用的侧信道信息，是密码算法在处理合法填充和不合法填充时会有所区分。与之前的方法不同的是，此时时间侧信道是因为调用 HMAC 算法次数的不同引起的。下面详细介绍一下这种攻击方法。

从 3.2 节可以得知，在 DTLS/TLS 协议中，程序解密密文后，首先会进行填充的合法性验证，然后基于解密后的明文，使用 HMAC 算法产生 MAC 标记，判断数据是否经过了篡改。HMAC 算法的如下：

$$MAC = Hash((opad \oplus key) \parallel Hash((ipad \oplus key) \parallel message))$$

上面的公式中，Hash 表示一种哈希算法，如 SHA1, SHA-256 等。*opad* 和表示外部密钥，*ipad* 表示内部密钥，二者的值是确定的。*key* 是在握手协议中协商好的用于 HMAC 算法的密钥，*message* 是需要进行处理的消息。最后产生的结果经过截断后，得到的结果就是需要的 MAC 标记了。需要注意的是，符号“ $\parallel$ ”表示两个字符串的连接，而不是逻辑上的与操作。公式中有两个 Hash 函数，根据所处位置的不同，称为外部 Hash 和内部 Hash。哈希函数的作用是对输入进行压缩，然后产生一个长度固定的独一无二的摘要。对输入的处理是分组进行的，如 SHA1 每次处理的数据长度为 64 字节。

假设 HMAC 算法中使用的哈希算法为 SHA1 算法，攻击原理如下。密码算法解密密文后，依旧进行填充检查，然后去除掉解密后的明文中消息验证码和填充部分，最后得到结果 *message1*（如果填充正确，那么 *message1* 就是真实的明文），这个明文字符串前会添加若干信息，如序列号、明文长度、版本等，得到的结果 *message2* 作为 HMAC 算法的输入（公式中的 *message*）。注意到，(*opad* $\oplus$ *key*) 的长度是 64 个字节，内层 Hash 的结果也是 64 个字节，因此，外层 Hash 函数总是要执行两次。内层 Hash 的内容由 (*opad* $\oplus$ *key*) 和 *message2* 拼接而成，前者的长度是 64 字节，后者的长度如果不是 64 的倍数，则需要填充，填充的最小长度为 9 个字节。如果 *message2* 的长度小于 64-9=55，则内层的 Hash 将会进行两次摘要处理操作，这时 HMAC 算法总共执行 Hash 压缩的次数就为 4。如果 *message2* 的长度大于 55，HMAC 算法总共执行 Hash 压缩的次数就至少为 5 次。

考虑解密一个构造好的密文（通过替换初始化向量），解密完成后的明文的长度为 64 字节，其组成为 *message1* $\parallel$ MAC $\parallel$ Padding，然后执行填充合法性检查，结果如下：

- 1，如果填充验证后是不合法，则算法会视填充长度为 0，最终 Hash 函数调用 5 次；

- 2, 如果填充的最后一个字节值为 0x00, 说明填充合法, 函数调用还是 5 次;
- 3, 如果填充的最后两个字节的结构为 0x01||0x01, 填充依旧正确, 函数调用为 4 次。

综上所述, 消息在处理过程中, 其长度的大小与调用压缩函数的次数是相关的, 而长度值由与填充的结构相关。因此, 验证填充的合法性会间接导致 Hash 函数的调用次数不同, 对外的表现就是, 接收端在收到密文后, 处理消息的时间并不均匀, 这就有了侧信道泄露的风险。因此, 攻击者可以精心构造特定长度的消息, 使得在程序在进行 MAC 计算的时候, 使用不同的时间, 然后从时间长短分析, 推断出填充的合法性验证结果, 也就是说攻击者成功获取了 Padding Oracle 的值。

3.4 本章小结

本章主要介绍若干侧信道的攻击基础, 这些内容可以后续章节提出新的攻击模型做一定的准备工作。

缓存攻击是一种高效的攻击手段, 无论是在对称加密领域, 还是在公钥加密领域, 它都扮演了非常重要的角色。这种思想甚至可以应用于除密码学之外别的领域中, 它值得研究者长期地研究。

DTLS 协议是为了弥补 TLS 协议的不足产生的, 它为进行安全的 UDP 会话提供了保障, 是一种重要的传输层协议, 目前国内对它的研究还不是很多。

Padding Oracle 攻击的关键点是在是否能获取 Oracle 的值。Vaudenay 在 2003 年首次提出的针对对称加密的攻击方案中, 使用时间差信息来区分填充检查失败和 MAC 校验失败者两种情况, 很自然地, 各个开源库使用“时间平衡”技术封堵了这个侧信道泄露。在之后的 10 年时间里, Padding Oracle 攻击一直没有实质性的进展, 直到 2013 年“Lucky Thirteen”这种攻击方法的提出, 随即也很快被修复。

本文第四章将提出利用缓存攻击获取 Oracle 的方法, 这种方法构建了获取 Padding Oracle 的新信道, 使得这种攻击方式重新焕发生机。

第四章 模型建立

本章首先从 AES 加密算法在存在的 CCA 安全缺陷入手，论证了展开 AES 攻击的可能性，使用数学语言描述了攻击的过程。接着引入了缓存攻击的方法，使得攻击过程具体化；最后分别从单台计算机和跨虚拟机两个方面，详细叙述了展开攻击的具体步骤。

4.1 CCA 攻击实验

由上文可知，AES 算法在 CBC 模式下是 CPA 不可区分安全的。因此，寻找新的针对 AES 加密算法的攻击模型，必须从 CCA 不可区分安全入手。首先考虑不可区分实验 $\text{PrivK}_{A,\Pi}$ 过程：

- 1，给攻击者带有参数 n 的输入 1^n ，攻击者 A 根据该输入输出一对明文 m_0 、 m_1 ；
- 2，使用密钥生成算法生成密钥 k ，然后从集合 $\{0, 1\}^n$ 中选取一个值 b' ，使用加密算法 Enc_k 加密明文得到密文 c ，然后将 c 交给攻击者 A ；
- 3，攻击者拿到密文后，使用自己的方法得到输出 b ；
- 4，如果 $b = b'$ ，则可表示为 $\text{PrivK}_{A,\Pi} = 1$ ，说明攻击成功，反之 $\text{PrivK}_{A,\Pi} = 0$ 。

如果规定 E 代表一个随机事件， $P(E)$ 表示该事件发生的概率，那么一个加密方案具备 CCA 不可区分安全也可以表述如下：

$$P(\text{PrivK}_{A,\Pi}^{CCA} = 1) \leq \frac{1}{2} + \sigma(n)$$

其中 $\sigma(n)$ 表示一个可以忽略的无穷小量， n 为该加密方案的安全系数。该式子表明，如果一个加密方案在 CCA 实验下的随机变量，其概率都可以规约到 $\sigma(n)$ ，那么这个加密方案是 CCA 安全的。

如果有一串需要使用分组密码进行对称加密的明文 m ，填充后的长度为三个明文分组，表示为 $m = m[0] \parallel m[1] \parallel m[2]$ ，填充策略可以考虑使用 DTLS 协议中的方法，即缺少 p 个字节则将这些位置填充为 $p-1$ 。加密时使用初始化向量为 IV ，密钥为 k ，使用 AES-CBC 加密方案加密后，得到的密文为 $c = c[0]c[1]c[2]$ 。加密的过程如下：

$$\begin{aligned} c[0] &= \text{Enc}_k(IV \oplus m[0]) \\ c[1] &= \text{Enc}_k(c[0] \oplus m[1]) \\ c[2] &= \text{Enc}_k(c[1] \oplus m[2]) \end{aligned}$$

使用 Dec 解密算法恢复明文的方法如下：

$$m[0] = Dec_k(c[0]) \oplus IV$$

$$m[1] = Dec_k(c[1]) \oplus c[0]$$

$$m[2] = Dec_k(c[2]) \oplus c[1]$$

需要注意，在把密文交给 Dec 解密的时候， IV 是和密文 c 一起传输的，且 IV 不需要加密。

使用上面所述的加密方案，考虑下面的攻击实验 $\text{PrivK}_{A,\Pi}^{CCA-POA}$ ：

1. 通过运行 $Gen(1^n)$ 生成一个密钥 k ；
2. 输入 1^n ，攻击者 A 可使用 Enc_k 和 Dec_k 。它输出一对长度相等的明文 m_0 、 m_1 ；
3. 从集合 $[1, 0]$ 中选取一个值 b' ，计算密文 $c = Enc_k(m_{b'})$ 并其提供给 A 。把 c 叫做挑战密文。
4. 攻击者 A 使用 Dec_k 若干解密操作，但直接解密挑战密文的方式还是不允许的。
 - i. 首先令 $IV' = IV \oplus temp_padding \oplus i$ ，将 IV' 和 $c[0]$ 作为输入提供给 Dec_k 。其中 $temp_padding$ 和变量 i 的长度均与 IV 长度一致，为分组密码的长度。 $temp_padding$ 的值取决于攻击目标字节在密文中的位置，以攻击密文最后一个字节为例，此时值 $temp_padding = 0x00 \dots 00$ ，而 i 的取值为 $0x00 \sim 0xFE$ ；
16-bytes
 - ii. Dec_k 获得输入后开始解密，解密后会执行填充检查，返回的结果有两种：a 填充（Padding）操作没有问题；b，填充有误。若返回的结果为 a，则继续恢复 $c[0]$ 的下一个字节，直到 16 个字节均被恢复；若为 b，则需要重新构造 $IV' = IV \oplus temp_padding \oplus i$ ，并提交给 Dec_k ；
 - iii. 依次完成 $c = c[0]c[1]c[2]$ 三个分组的解密过程；
 - iv. 最终 A 输出一个比特 b' 。
5. 如果 $b = b'$ ，实验的输出为 1，意味着攻击成功，否则输出为 0。

攻击实验 $\text{PrivK}_{A,\Pi}^{CCA-POA}(n)$ 建立与 CCA 不可区分攻击实验 $\text{PrivK}_{A,\Pi}^{CCA}(n)$ 实验的不同在于前者改进了攻击者 A 和 Dec_k 交互的方式，第 4 步的过程可以图 4.1 表示：攻击者构造好密文，然后交给算法进行解密，算法解密后首先会检查填充的合法性，然后把结果反馈给攻击者，攻击者根据这个信息的正确与否，决定是继续构造还是恢复明文。需要注意的是，攻击者获取到的 *Oracle* 是解密算法主动反馈给攻击者的。

事实上，攻击实验 $\text{PrivK}_{A,\Pi}^{CCA-POA}$ 总是可以获得成功的，原因是在实验中的 4(ii) 步中，如果在加密前的填充操作是正确的话，得到的 $m[0]$ 的最后一个字节必须是 i ，原因如下：

$$\begin{aligned}
m[0] &= Dec_k(c[0]) \oplus IV \oplus temp_padding \oplus i \\
\Rightarrow m[0] &= Dec_k(c[0]) \oplus IV \oplus temp_padding \oplus i \\
\Rightarrow m[0] &= m[0] \oplus temp_padding \oplus i \\
\Rightarrow m[0] &= temp_padding \\
\Rightarrow m[0] &= \underset{16\text{-bytes}}{0x00...0000}
\end{aligned}$$

因此,通过固定 $temp_padding$ 且迭代 i 的方法,可以获得明文第一个分组最后一个字节的值。类似的,如果要恢复第二个字节,需要设置 $temp_padding$ 的值为 $\underset{16\text{-bytes}}{0x00...0101}$ 。通过不断的尝试, $m = m[0] \parallel m[1] \parallel m[2]$ 可以完全恢复出来。若整个明文分组有 l_m 个字节,则使用上述方法恢复明文的算法复杂度为 $\frac{1}{16*16*l_m} = \frac{1}{256*l_m}$ 。

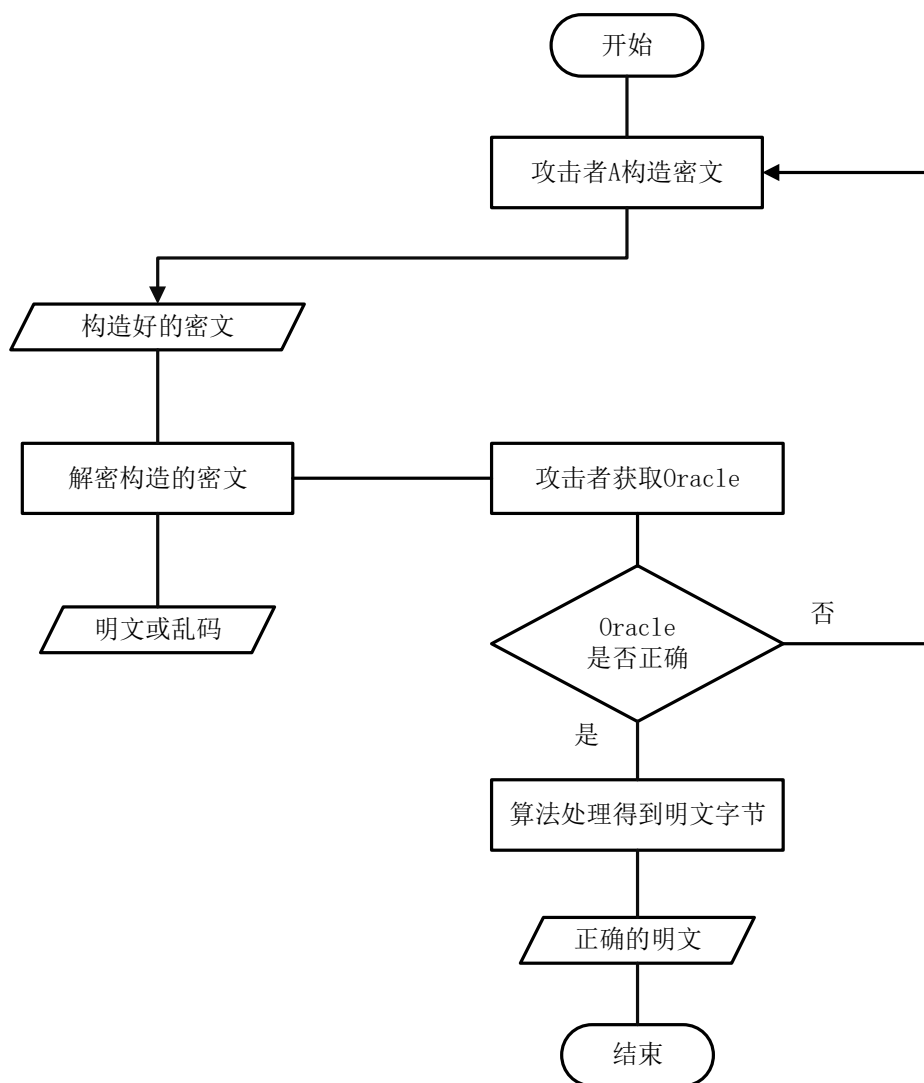


图 4.1 攻击者 A 恢复明文的过程

攻击实验 $\text{PrivK}_{A,\Pi}^{\text{CCA-POA}}$ 是建立在不可区分性的基础上的。规定获取明文是否填充正确这个操作为随机事件 B，通过 Padding Oracle 信息来判断 $b = b'$ 是随机事件 C。则在 $\text{PrivK}_{A,\Pi}^{\text{CCA-POA}}$ 不可区分攻击实验中，攻击成功的概率可表示为：

$$P(\text{PrivK}_{A,\Pi}^{\text{CCA-POA}} = 1) = P(BC)$$

其中， $P(BC)$ 表示获取到 Padding Oracle 信息后，通过迭代恢复明文，然后判断出 $b = b'$ 的概率的大小。

在实验的第 4 步中，获得 Padding Oracle 信息与恢复出明文是两个独立事件。并且，如果已经获取到了 Padding Oracle 信息，进行明文的恢复是一个确定性事件（只需要进行异或操作），即 $P(C) = 1$ 。所以， $P(BC) = P(B) * P(C) = P(B)$ ，攻击成功的概率可写作：

$$P(\text{PrivK}_{A,\Pi}^{\text{CCA-POA}} = 1) = P(B)$$

上面的式子表明， $\text{PrivK}_{A,\Pi}^{\text{CCA-POA}}$ 攻击取得成功的关键是：引入的随机事件 B 的概率满足

$$P(B) > \frac{1}{2} + \sigma(n), \text{ 从而使得 } P(\text{PrivK}_{A,\Pi}^{\text{CCA-POA}} = 1) = P(B) > \frac{1}{2}。$$

综上所述，这种建立在 CCA 安全缺陷上的攻击方案，如果必须具备下面两个条件，是可以攻击成功的：条件 1，可以获取到密文解密后的 Padding Oracle 信息；条件 2，获取 Padding Oracle 信息的概率必须大于 1/2。但是，在现有的 AES-CBC 算法的各种实现（如 OpenSSL、GnuTLS、NET）中，Padding Oracle 信息是无法获取到的，该信息或是被加密，或被隐藏。因此，条件 1 被破坏后，攻击者已经不能通过主动向密码算法询问来获得 Padding Oracle 信息，显然后续也无法继续展开攻击。为了使得攻击继续有效，在下一节中，本文将介绍一种侧信道获取 Padding Oracle 的方式，同时这种手段作为一种随机事件，其概率也满足条件 2 的要求。

4.2 缓存攻击模型

本文引入缓存攻击来获取 Padding Oracle 信息。算法在执行时，缓存攻击通过监测相关指令在缓存中的变化，推断出密码算法的执行流，最终得到有用的信息。因为监视的目标为指令，因此具有“手术刀”般的精准性，与之前的方法^{[25][26]}相比，具有更高的成功率和攻击效率。缓存攻击这个随机事件满足 $P(B) > 1/2$ 的要求，可以达到利用 CCA 安全缺陷攻击 AES-CBC 加密方案的目的。

图 4.2 为本文提出的引入缓存攻击后的攻击模型示意图，它建立在攻击实验 $\text{PrivK}_{A,\Pi}^{\text{CCA-POA}}$ 的基础上。模型主要由三个模块组成：加解密模块，缓存攻击模块以及数据处理模块，其简要的攻击流程是：在使用加解密模块对密文进行解密的过程中，使用缓存攻击模块展开攻击，得到 Padding Oracle 信息，如果 Oracle 信息错误，则继续构造密文；如果信息正确，则将其输入最后的数据处理模块，可以得到最后的部分明文或全部明文。其中最重要的模块是缓存攻击部分，现有已被提出的基于 Padding Oracle 攻

击方法，利用了不同的侧信道信息，不过均被一一修复了。如第三章介绍的最新攻击构造，就是利用不同密文可能会执行不同次数的 HMAC 操作，导致时间侧信道泄露这一点来获取 *Oracle* 信息，修复的手段就是强制调用相同次数的 HMAC 算法。缓存攻击模块的引入，使得构造新的攻击手段得以实现。

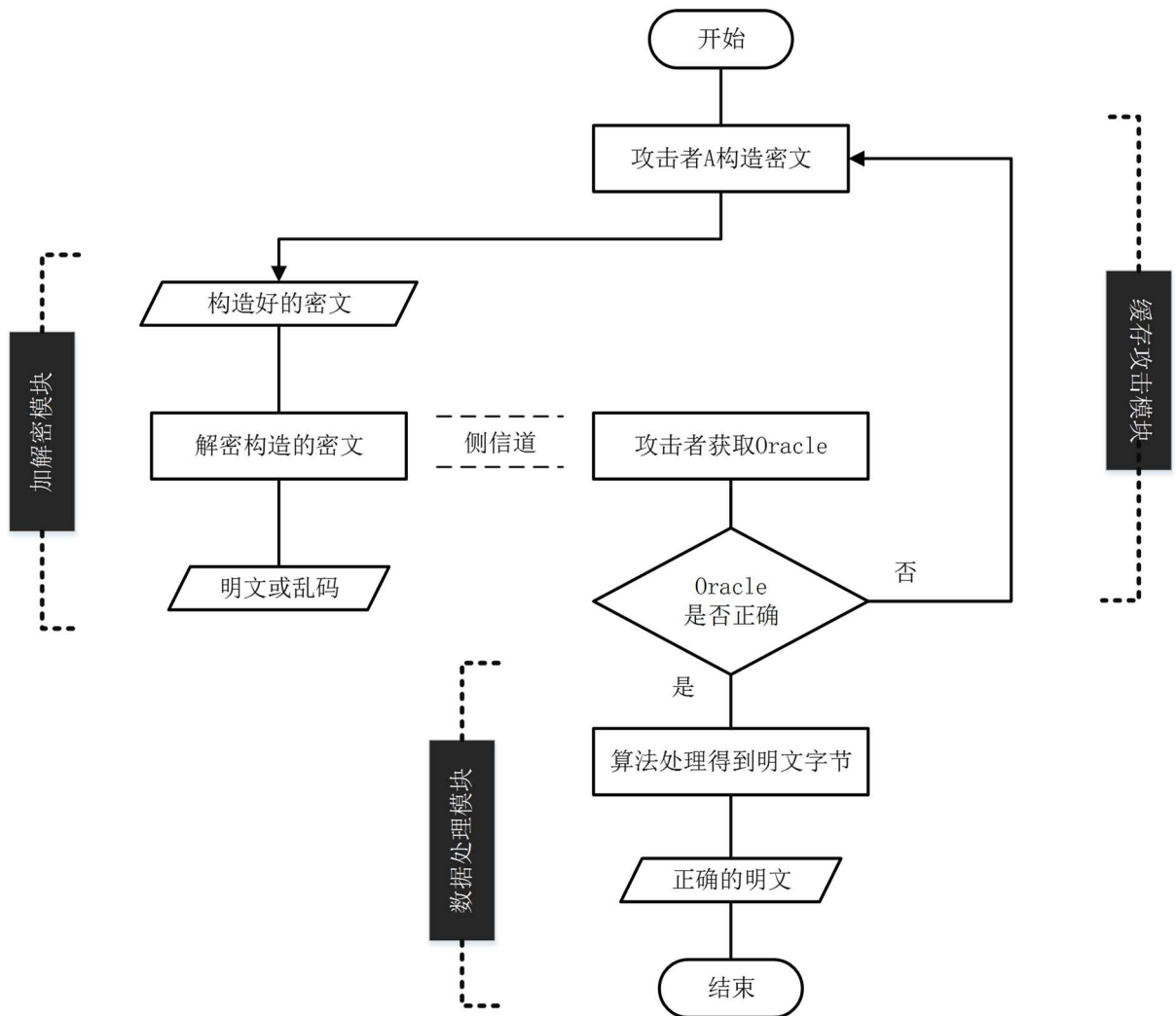


图 4.2 基于 CCA 缺陷的缓存攻击模型

4.2.1 加解密模块

模型中的加解密模块是基于 GnuTLS 库中的 DTLS 协议实现的。DTLS 协议位于 TCP/IP 协议栈的传输层和应用层之间，它在 RFC4347 中被提出，用来保证敏感延迟应用，如实时视频会议、Internet 电话和 VPN 等的安全性，因为这些应用程序都是基于不可靠的 UDP 进行传输的。DTLS 协议拓展了现存的 TLS 协议架构，使之支持 UDP 传输，即成为 TLS 的一个支持数据报传输的新版本，并保持了和 TLS 相同的架构。

DTLS 协议由握手协议、改变密码规格协议、警告协议、应用层数据协议和记录层协议组成，其中的记录层存在加解密模块。鉴于此，本文中的加解密模块使用了其中的

记录层协议，去除了冗余的且与实验关联不大的握手协议与警告协议等，保留了最为关键的加解密功能。

加解密模块的主要作用是执行加解密操作，攻击实验中的加解密算法是 AES，密钥长度为 256 位，加密模式为 CBC(Cipher Block Chaining，分组链接模式)。同时，加密模块还具有 HMAC 校验的功能，用来保证数据的完整性，攻击实验中使用的 HMAC 校验算法为 SHA-256。

4.2.2 数据恢复模块

数据分析模块的作用是获取攻击模块得到的“Oracle”信息后，根据 AES 加密算法的流程，计算出密文对应的明文。

表 1.3 所示为恢复一个分组密文的算法伪代码。其中的语句 *if (Oracle (D (Cb, temp_IV)) = VALID)* 是为了判断在输入 *temp_IV* 时，是否返回正确的“Oracle”，即是否填充正确了，这条语句的功能由缓存攻击模块来完成。从算法中也可以看出，恢复一个字节的明文，需要构造 256 次密文，更准确地，是需要将初始化向量 *IV* 的某个字节从 0-255 进行迭代，然后和密文一起再次进行解密。

数据恢复模块是整个攻击过程的最后一步，其过程可以由确定性的算法描述。因此，数据恢复模块的工作不影响整个攻击模型的攻击成功率，只要能正确获取 Padding Oracle，则一定能恢复出明文来。

表格 4.1 数据恢复模块的算法描述

Input: C_b, IV
Output: P_b -- b -byte plaintext
 Let $temp_IV$ and $median$ be array of b bytes.
begin
 for $i=0$ to $b-1$ **do**
 for $byte = 0$ to 255 **do**
 $temp_IV[i] = byte$
 if $(Oracle(D(C_b, temp_IV)) = VALID)$
 $Median[i] = byte \oplus temp_IV[i]$
 $P_b[i] = median[i] \oplus IV[i]$
 break;
 for $j = 0$ to i **do**
 $temp_IV[j] = median[j] \oplus (i+1)$
 end
Oracle (D (C_b, temp_IV)) denotes that decrypting ciphertext C_b with temp_IV and returning the oracle.

4.2.3 缓存攻击模块

缓存攻击模块利用了缓存攻击这种侧信道技术，解决了在目前的 AES 算法实现中无法获取到 Padding Oracle 这个难题。本模块的攻击目标是加解密模块，通过监测特定指令在加解密模块中的执行轨迹，推断出算法的执行流，最终得到解密后的 Padding Oracle 值。

加解密模块使用的是 DTLS 协议的部分内容，该协议用来加密两个 UDP 传输端的数据安全。数据在使用 DTLS 协议传输的时候，并不是简单的将明文消息使用对称密钥加密，而是采用了一种更为复杂的过程（见图 3.3），用来保证数据的安全和完整。明文首先会被加上一个“消息头”，这个“消息头”包含此次会话（Session）的一些特征信息，如此次会话的序号。加上“消息头”的明文紧接着会通过一个 HMAC 算法进行计算，得到一个独一无二的 HMAC 值，上述三个字符串通过拼接后，得到一个新的明文。但这个新的明文的长度往往并不是分组长度的倍数，因此需要进行填充操作，填充后的明文称为最终明文，这个最终明文经过对称加密算法进行加密后，就可以进行安全地传输了。通信的另一端接收到密文后首先解密，然后检查填充是否合法，最后再将解密后的明文通过 HMAC 算法进行校验，判断消息是否经过了篡改。

表 4.1 所示的是开源库 GnuTLS（版本为 3.13）中解密密文后，执行 Padding 合法性判断和 HMAC 校验的代码实现。`data[]` 数组储存了解密后的结果，在 652 行，数组的最后一个字节赋值给了变量 `pad`，由第三章相关知识可知，`pad` 的值就是进行填充操作时，重复填充的字节的大小。获取到 `pad` 后，程序在 658~678 行进行 `pad` 值的校验，验证其是否满足填充的规则，然后将结果赋值给变量 `pad_failed`。如果检查后出现了错误的填充，`pad_failed` 会设置为 1，否则被置为 0。677 行计算真实明文的长度，方法是将解密结果中的 HMAC 校验值和 Padding 值移除。

在早期版本的 OpenSSL 和 GnuTLS 中，当填充不合法的时候，程序是不会进行完整性校验的，因为明文既然存在不合法的填充，那么计算出来的 HMAC 肯定也是错误的。但考虑到在进行完整性校验时，计算 HMAC 这个过程是一个耗时的操作，因此，这种处理明文的方式就暴露出了存在侧信道攻击的危险，Canvel 提出的攻击方法就利用了这个原理。鉴于此，程序在 691 行计算了 HMAC 的数值——不管 `pad_failed` 是 1 还是 0，然后把计算的结果赋值给了变量 `tag`。722~725 行显示出程序需要比较计算所得的 HMAC 值 `tag` 和加密前存储的 HMAC 值 `tag_ptr`，比较失败后会返回一个 1。如果比较失败，或者出现填充合法性校验失败，程序会进入一个 `dummy_wait` 函数。开源库中该函数的目的是为了挫败另一种时间侧信道攻击：攻击者构造精巧的 DTLS 包，通过微小的时间差，区分“错误 HMAC+合法 Padding”与“错误 HMAC+不合法 Padding”这两种情况，从而达到获取 Padding Oracle 的目的。在本文的攻击方法中，因为与解密后中间值进行异或操作的 IV 经过了重新的构造，因此在每轮攻击中，HMAC 的比较都会失败，程序每次都会调用 `dummy_wait` 函数。

当程序跳入 `dummy_wait` 后，448 行会进行一个条件语句分支的判断，决定下一步是否进行冗余的哈希计算。判断语句中的条件有两个：填充合法性校验的结果 `pad_failed` 以及填充 `pad`，更明确地说：条件分支语句内的代码块被执行当且仅当 `pad_failed > 0` 及 `pad > 0`。回到代码的 657~675 行，当合法性校验失败的时候，`pad_failed` 和 `pad` 会分别被设置为 1 和 0（另一种情况是明文的长度太短，这里不考虑），因此，可以通过监测 449~470 行代码的执行情况，来反推 `pad_failed` 的值，即攻击者所需要的 Padding Oracle。

表格 4.2 GnuTLS 中解密代码片段

```

652   pad = compressed->data[ciphertext->size - 1];
...
659   for (i = 2; i <= MIN(256, ciphertext->size); i++) {
660       tmp_pad_failed |= (compressed->data[ciphertext->size - i] != pad);
663       pad_failed |= ((i <= (1 + pad)) & (tmp_pad_failed));
665   }
667   if (unlikely(pad_failed != 0 || (1 + pad > ((int) ciphertext->size - tag_size)))) {
673       pad_failed = 1;
674       pad = 0;
675   }
677   length = ciphertext->size - tag_size - pad - 1;
...
691   ret = _gnutls_auth_cipher_tag(&prams->read.cipher_state, tag, tag_size);
...
722   if (unlikely(memcmp(tag, tag_ptr, tag_size) != 0 || pad_failed != 0))
725       dummy_wait(prams, compressed, pad_failed, pad, length + preamble_size);
.....
437   void dummy_wait (...) {
...
448   if (pad_failed == 0 && pad > 0) {
449       len = _gnutls_mac_block_size(prams->mac);
450       if (len > 0) {
454           if (...) {
456               if (len < plaintext->size)
457                   gnutls_auth_cipher_add_auth (...);
461           else
462               gnutls_auth_cipher_add_auth (...);
472   }   }   }

```

总结一下上面的代码工作流程，可以得到一个攻击思路：通过监测代码 449~470 行中某一指令（这几行代码对应着几百条指令）的执行情况，可以得到 **Padding Oracle** 信息。如果该指令没有被执行，说明填充正确，如果该指令被执行了，说明填充的合法性校验失败。在进行实际攻击的时候，称缓存攻击所在的进程为攻击进程，加解密模块所在的进程为目标进程，则具体的攻击流程如下（假设被监测的指令为 A）：

- 1，攻击进程将目标进程的 ELF(Executable and Linkable Format)文件映射到自己的进程空间中；
- 2，一轮攻击实验开始（见图 4.3）：
 - a) 攻击进程使用 *clflush* 指令驱逐指令 A 所在的缓存 line，等待目标进程工作；

- b) 目标进程接受构造好的 *IV* 和密文，执行解密操作，此时攻击进程等待；
 - c) 攻击进程再次读取指令 A，若读取指令 A 的时间较长，说明指令 A 在攻击进程等待的周期内，没有被访问，若时间较短，则说明指令 A 被访问了。
- 3, 若指令 A 被访问了，则说明构造的密文使得 **Padding** 合法性校验正确，此时将数据交给数据恢复模块，可以恢复出一个字节的明文。否则跳转至步骤 2，重新进行下一轮的 *IV* 构造及攻击实验；
- 4, 所有密文分组处理结束后，攻击结束。

至此，缓存攻击模块的设计工作就完成了，该模块满足了通过侧信道获得 **Padding Oracle** 的要求，并且具有很高的获取精度（实验的结果为 99.5%）。

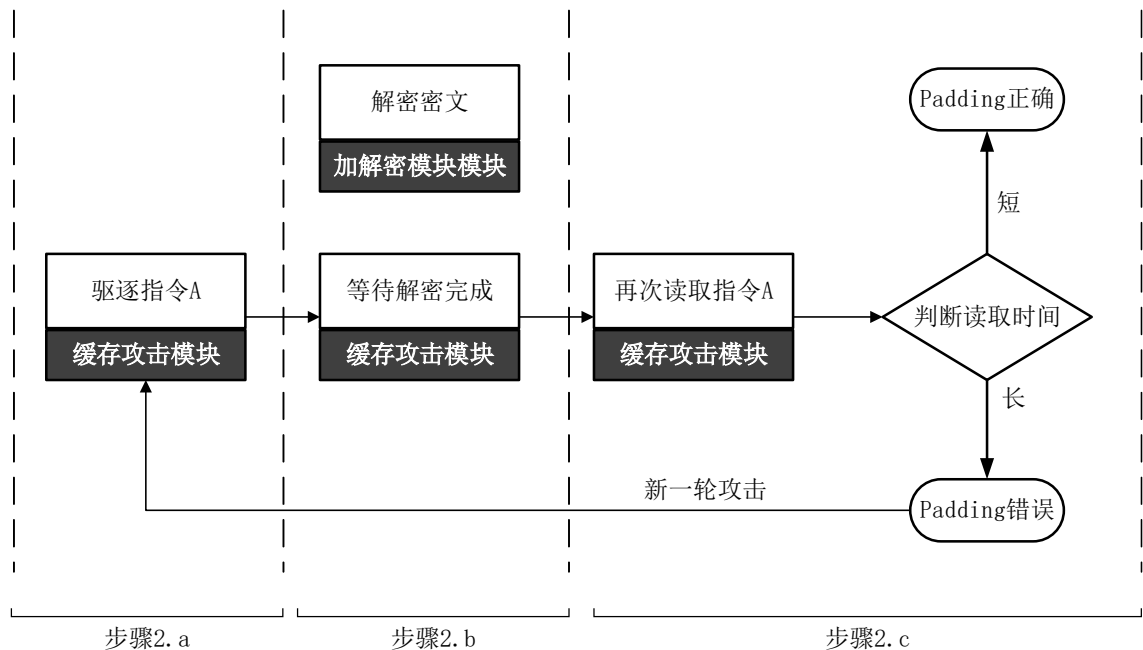


图 4.3 缓存攻击模块核心流程

4.3 单台计算机攻击实验

使用图 4.2 中的攻击模型，可以对使用 AES-CBC 模式的加密过程展开攻击，整个攻击过程在单台计算机上完成，攻击的双方分别处于两个进程中，进程使用的指令和数据共享 L3 Cache。考虑到使用 CBC 链接的对称加密算法，都存在 CCA 缺陷，因此一般化的攻击实验设计步骤可以分为以下三个部分：功能模块划分、确定攻击位置、调整环境参数。

4.3.1 功能模块划分

根据图 4.2 中的攻击模型，密文构造、缓存攻击模块、数据恢复模块被放在一个进程中，即攻击进程。加解密模块所在的进程是攻击的对象，称为目标进程。攻击进程的功能是不断给目标进程发送篡改后的密文，同时使用缓存攻击检测目标模块的执行，如果获取到了 *Oracle* 后，使用算法恢复明文。目标进程可以被视为黑盒，它只负责数据的

加解密操作。

攻击进程 A 和目标进程 B 的功能代码由 C++ 实现，各自编译后，得到 ELF 可执行文件。在攻击之前，需要使用 *mmap* 命令能将目标进程的 ELF 文件映射到攻击进程的虚拟地址空间中。两个进程工作后，其内存空间的布局如下图 4.4 所示。

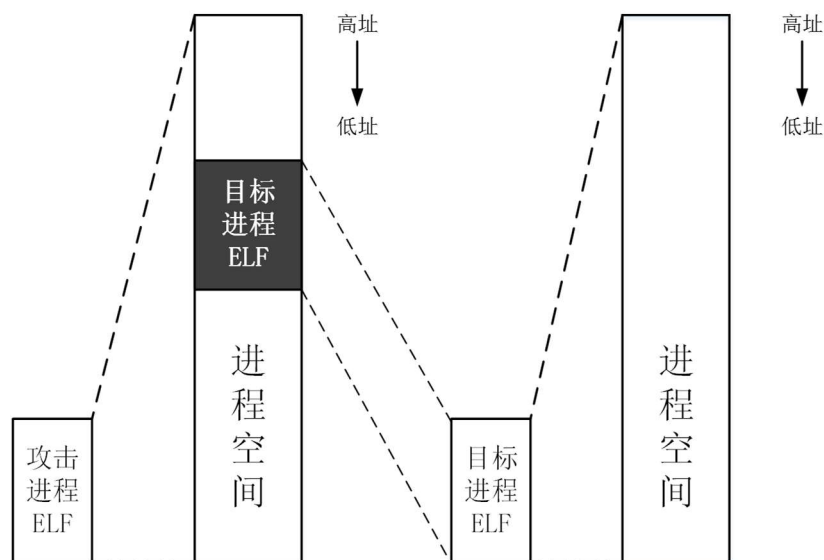


图 4.4 攻击进程和目标进程的进程空间分布

上图中，攻击进程和目标进程拥有各自独有的进程空间，同时因为攻击进程将目标进程的 ELF 文件被映射到了自己的共享空间内。因此，磁盘上目标进程的 ELF 文件就同时存在于两个进程中，如果攻击进程将共享空间中的指令从缓存中驱逐，则它会直接体现在目标进程相应的指令也被驱逐。

4.3.2 确定攻击位置

攻击进程和目标进程的功能完成后，需要确定攻击的位置。在 4.2 节中，可以通过分析 GnuTLS 库中 DTLS 协议的解密过程，最后确定需要检测的指令位置。通过攻击这些位置，可以得到 Padding Oracle 的值。事实上，不仅仅是 GnuTLS 库，开源库 PolarSSL, GyaSSL 中，同样存在可以获取 Oracle 的侧信道，因此，图 4.2 的攻击框架可以适用于多个攻击目标。

分析开源库的源码得到需要监测的指令或函数后，还需要得到该指令或函数在进程空间中的具体位置，使用一些逆向工程的手段可以实现这一目标。本文的实验中，因为攻击进程和目标进程都是基于开源库实现的，因此在编译后可以选择编译调试工具（如 Linux 下的 gdb）得到指令的位置。如果待攻击的目标已经无法调试，可能还需要借助反汇编的手段，如 ida pro 这个工具就是很好的选择。需要注意的是，实际攻击中使用的指令地址，还需要在反汇编得到的地址上，加上攻击进程中共享空间的基地址。

4.3.3 调整环境参数

在进行攻击实验之前，还需要注意的实验环境对攻击效果的影响，这些环境因素甚至决定着实验是否可以取得成功。

第一个对攻击效果有重要影响的就是预存取技术（Pre-fetch）。x86（以及其他很多体系结构）的 CPU 提供了 pre-fetch 系列指令，用于将指定地址的内存预取到缓存。一个典型的例子就是线性存储的数组，CPU 在使用当前数组元素时，会提前将相邻若干个数组元素读取到缓存中，避免 CPU 直接从内存中读取，减小取数据的时钟周期。由于缓存攻击模块是监测的是指令是否在一段时间内被使用，判断的依据就是该指令是否被读取到缓存中。因此，如果被监测指令和需要被频繁执行的指令分布在缓存中非常近的位置，Pre-fetch 技术就会给攻击带来一定的困扰。可以选择关闭这一项功能，如某些系列的计算机可以在开机的 BIOS 中选择禁用 Pre-fetch，这样可以给攻击带来更高的实验精度。

第二个需要注意的是实验中的进程噪声。攻击实验中同时运行了两个进程，攻击进程是以一定的周期读取缓存中的数据，因此，如果存在其他的进程干扰，CPU 就会暂停当前的进程，改为执行其他的进程，这将为实验引入误差。实验前，最好是关闭与实验无关的其他进程，如在 Linux 下，可以使用 kill 命令完成这一操作。

上面三个步骤完成后，即可同时运行攻击进程和目标进程，完成攻击实验。这里可能需要用到若干 shell 命令来实现两个进程的启动和通信。

4.4 跨虚拟机攻击实验

虚拟化技术是云服务得以实现的核心技术，因为虚拟化技术能够将云的基础架构平台的相关硬件资源进行虚拟化后提供云服务，它在提高硬件平台的利用率、稳定性、安全性等方面得到越来越广泛的使用。但与此同时，虚拟化带来的安全性问题也不容小觑。因为云是同时为多个用户提供服务的，来自不同组织的用户可能在一个云平台，共享硬件设备，这就为侧信道攻击提供了可能。

本文提出的攻击方法需要两个程序在共享同一缓存，即使用相同的硬件平台，此特征符合了云的特点。虚拟化的主要作用就是使得本来是在不同的操作系统上、不同的硬件平台运行的程序可以在一台电脑主机上运行，共享物理主机上的硬件资源。

因此，单台计算机上的攻击方法，可以很顺利地迁移到不同的虚拟机上，即跨虚拟机的攻击。这种新的攻击中，模块划分、攻击位置确定和环境参数确定，还是和单台计算机上的攻击方法的保持一致，唯一不同的是，此时攻击进程和目标进程同时运行在不同的虚拟机上，因此两个虚拟机需要共享一个文件夹，来实现文件的映射。如图 4.5 中所示，单台计算机攻击实验，两个进程处于同一操作系统下，共享同一硬件平台；当把攻击模型应用到跨虚拟机的情形时，两个进程处于不同的两个操作系统，具有不同的内存划分，独立工作不相干扰，但在这种环境中，两个进程还是使用的相同的硬件（这里

的硬件主要是指 CPU 和缓存)。

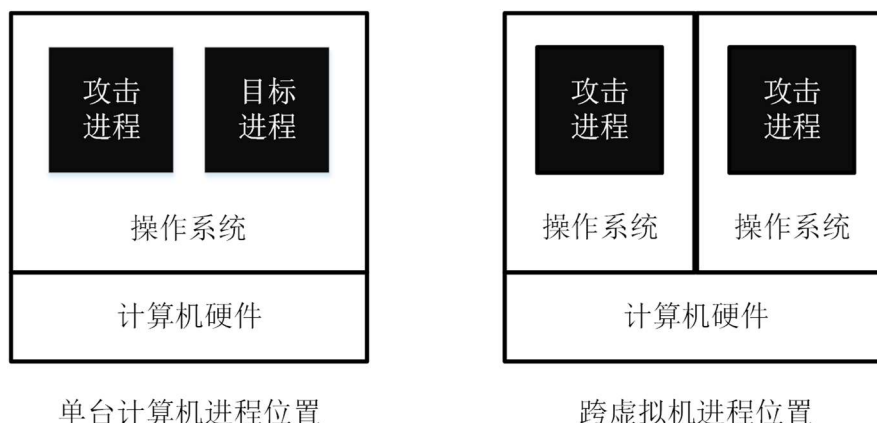


图 4.5 单台计算机和跨虚拟机两种情形中进程的位置

实验可以使用 VMware 模拟云平台，同时运行两个 Ubuntu 虚拟机，让目标进程和攻击进程分别运行在这两个虚拟机上，两个进程共享相同的硬件设备。在实际的云下，如何使攻击进程和目标进程处于同一硬件平台下，对攻击者是一个障碍，这个问题也被称作“共驻(co-location)”。2009 年，Ristenpart 等^[8]的论文证明，可以使用“云制图”的网络扫描策略，得到了基于云的 VM 位置，然后放置一个恶意 VM 在相同的物理平台上，随后访问共享硬件和缓存位置，执行各种侧信道攻击。因此，本文跨虚拟机的实验设计虽然是对实际攻击的模拟，但也可以给实际的云中的攻击提供一定的借鉴。

4.5 攻击效率分析

在上述的两个攻击实验设计中，攻击目标均为恢复单个密文分组的最后一个字节。一个字节的恢复需要攻击者提交篡改的密文 256 次进行解密，一次解密中，攻击者的监测活动可能会受到环境噪声的影响。

噪声源主要来自三个方面。第一个噪声源为除了攻击进程和加密进程外的其他运行中的进程，这些进程在执行的时候也需要访问内存，同时将指令或者数据加载到缓存中，这会给攻击进程的监测活动带来干扰。第二种噪声来自于处理器的预存取 (pre-fetching) 功能。CPU 会通过推测后续指令和数据的使用规律，将数据提前加载到内存或缓存中，这可以带来性能上的提升，但与此同时也给监测活动引入了误差。最后一个噪声是由加密方案本身引起的。在加解密过程中，不同的操作耗费时间略有不同，由于本文使用固定时间窗口进行指令监测，因此时间的差异将给监测活动带来一定的干扰，即驱逐指令后再读取指令的时刻，有可能会提前，也有可能会滞后，这将影响判断缓存的命中与失效。上述的三种噪声源中，都实验结果影响最为显著的就是第一种进程干扰噪声了，它会直接影响实验中是否可以得到准确的 Padding Oracle。对于第二种噪声，攻击者可以关闭硬件的预存取从而避免。第三种噪声对攻击实验的影响很小，并且现有的各种安全

厂商使用了时间平衡的方法来抵抗基于时间泄露的侧信道攻击，因此不同操作带来的时间误差和一次解密用的总时间相比可以忽略不计。因此，衡量攻击模型最终的攻击效率，主要考虑第一种噪声。

从上面的缓存攻击模型可以得知，攻击实验是否可以取得成功，最终达到可区分的目的，需要随机事件 B （获得 Padding Oracle）满足以下关系：

$$P(\text{PrivK}_{A,\Pi}^{\text{CCA-POA}} = 1) = P(BC) = P(B) = \frac{1}{2} + \sigma(n)$$

式子中的 $\sigma(n)$ 表示一个大于 0 的无穷小量， n 表示密码方案的安全系数。上面的公式表明，如果攻击者准确获取 Padding Oracle 的概率大于 0.5，则密码方案就是不安全的。定义攻击者在一次获取 Padding Oracle 的时间里，进程干扰噪声引起的缓存失效现象为事件 N ，其概率大小为 $P(N) = p_{\text{miss}}$ 。则可以使用全概率公式得到存在噪声干扰时攻击者的攻击效率为：

$$\begin{aligned} P(B) &= P(\overline{BN} + BN) \\ &= P(\overline{BN}) + P(BN) \\ &= P(\overline{N}) * P(B | \overline{N}) + P(N) * P(B | N) \end{aligned}$$

其中 $P(B | \overline{N})$ 和 $P(B | N)$ 是事件 B 的条件概率，分别表示在缓存失效和命中的时候获取到准确 Padding Oracle 的概率。由攻击的原理可知，256 次构造的密文中，只会出现一次正确的 Padding Oracle，此时某些指令被使用，攻击进程再次读取该指令的时候，只需要较少的时钟周期。如果某一次提交的密文，对应着正确的 Padding Oracle，但是由于其他进程的干扰，使得待监测指令从缓存中被驱逐了出去，攻击者就不能得到正确的结果，这中情况就对应着上述的事件 N ， p_{miss} 就可以进一步地表示缓存的失效率（Cache Miss Rate）大小。因此， $P(B | N)$ 就表示此时对应的 Cache line 已经不在缓存中了而攻击者还能获取到正确结果的概率，这种情况当然是不可能发生的。相应的， $P(B | \overline{N})$ 表示攻击者可以顺利获得 Padding Oracle 的概率，由于这里只考虑了第一种噪声，故 $P(B | \overline{N}) = 1$ 。因此，攻击效率可以进一步写作：

$$P(B) = P(\overline{N}) * P(B | \overline{N}) + P(N) * P(B | N) = 1 - p_{\text{miss}}$$

从上式主要可以得到两条重要的信息。其一，缓存攻击模型最终的攻击效率既可以用缓存的失效率来衡量；其二，缓存的失效率需要满足一定关系，才可以使得 $P(B) > 1/2$ ，从而达到不可区分的效果。

本文提出的缓存攻击模型获取 Padding Oracle 的原理，是通过监测两个进程共享的指令缓存变化，推断出准确的结果。监测的共享指令，是存在 L3 Cache 中的，由缓存的结构可以知道，从 L3 Cache 到 L1 Cache，缓存的失效率逐步增加。其中 L2 Cache 的缓存失效率(Cache Miss Rate)大约为 L3 Cache 的 4 倍，L1 Cache 大约为 L3 Cache 的 6 倍，因此 L3 Cache 的低缓存失效率，就可以为模型提供较高的攻击效率。目前已经有若干

模型来估算不同的缓存结构下,缓存失效率的变化和分布情况,如文献中^[37]使用马尔可夫链建立了一个统计模型,用来估算应用程序在运行过程中数据在缓存中的失效率情况。文献^[36]中给出的数据显示,L3 Cache 的平均失效率为 1%,最大可以达到 4%,数据的结果是由标准性能评估公司 SPEC 推出的 CPU 子系统评估软件 SPEC CPU 2006 计算得到的。

综上所述,建立在 L3 Cache 上的指令监测攻击进程,其攻击成功率与 L3 Cache 的缓存失效率密切相关。攻击的效率估计为 $P(B) = 1 - p_{miss} = 1 - 1\% = 99\%$,这个结果是一个平均值,同时也是忽略的其他噪声影响。因此,从噪声估算的角度,本文提出的攻击模型是可以达到攻击目的,后文会证明这一点。

4.6 本章小结

本章基于 CCA 安全缺陷,首先描述了一个针对 AES-CBC 加密方案的攻击实验,该实验说明,攻击者使用一定的构造技巧,可以最后达到区分明文的目的,同时该攻击实验也得到了实验成功所需要的概率要求:获取 Padding Oracle 的概率大于 1/2。

图 4.2 的缓存攻击模型建立在之前攻击实验的基础上,该模型主要解决了无法获取 Padding Oracle 的难题。同时,以 GnuTLS 开源库为例,分析了实现这种缓存攻击的技术细节。攻击模型虽然是以 AES-CBC 模式论证的,但事实上,任何使用 CBC 模式的对称加密算法,都可以使用这个模型进行攻击,因为它们都存在 CCA 安全缺陷。

最后,本章从实际的攻击出发,详细叙述了攻击模型适用的两种场景:单台计算机下的攻击和跨虚拟机的攻击。

第五章 实验结果

使用第四章中的攻击模型，本文对 AES-CBC 模式的解密过程进行了攻击实验。在实验开始前，首先进行一次缓存的行为特征探究，佐证了缓存攻击的能力。实际的攻击场景分为两种，一种是在单台计算机上，目标进程和攻击进程运行在同一个操作系统下；另一种是跨虚拟机（cross-VMs）的攻击，两个进程分别运行在不同的虚拟机（操作系统）中。本章最后分析了实验的结果，并指出了实验存在的一些问题。

5.1 环境参数

在本文两个攻击实验中，攻击的目标都是一致的：使用攻击进程监视目标进程的解密活动，得到单个字节或者全部分组的明文。实验中，目标进程是基于 GnuTLS 中的 DTLS 协议部分实现的，使用的加密算法是 AES，加密操作模式为加密分组链接（CBC），密钥长度为 256 位。DTLS 协议中，因为有消息验证码（MAC）的生成环节，因此本试验中使用 HMAC 算法，其中的 HASH 算法为 SHA256。

待监测指令的确定是分析 Padding Oracle 在解密时的生成过程得到的。编译好目标进程的 ELF 文件后，本文实验使用利用反汇编得到了指令在二进制文件中的偏移位置，此位置的数值加上目标进程 ELF 在攻击进程空间中的基址，得到的结果就是在实际攻击中使用的监测位置。

表格 5.1 攻击实验的环境参数

条目	实验参数
CPU	Pentium(R) Dual-Core E5500
Architecture	X86-32bit
RAM	4GB
Clock Speed	2800MHz
L1 data cache size	32KB
L1 instruction cache size	32KB
L2 cache size	2048KB
Cacheline size	64B
Operating System	Ubuntu 14.10

攻击实验运行的实际硬件环境如表 5.1 所示。计算机具有两级缓存，其中 L2 Cache 被所有进程所共享，cache line 的长度为 64 字节，进程运行在 Linux 操作系统下，具体版本为 Ubuntu 14.10。为了减小试验中 Pre-fetch 实验结果造成的影响，计算机 BIOS 中硬件的预取指令功能被选择关闭。

5.2 攻击实验结果

5.2.1 缓存行为特征实验结果

程序运行需要将频繁使用的指令从内存读取到缓存中，以便 CPU 能方便的访问。当 CPU 需要的指令位于缓存中时，CPU 可以立即读取，需要的时间较少，这种缓存行为称之为命中；而当这些指令不在缓存中，则处理器必须从主存中读取指令，耗费的时间较长，这种缓存行为称之为未命中（失效）。

缓存攻击模块是攻击实验中的核心，它利用的就是 CPU 取数据时缓存出现的失效/命中现象。因此在正式的实验之前，很有必要验证缓存的这一行为特征。

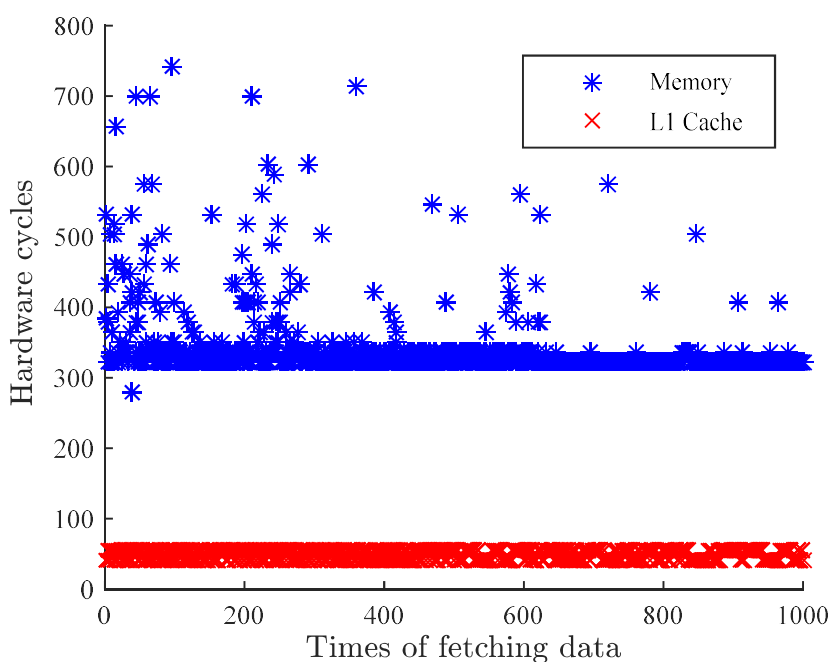


图 5.1 缓存的行为特征

图 5.1 中的蓝色点表示 CPU 从内存取数据/指令的时间，红色表示从缓存中取数据/指令的时间，这两种情况分别对应着缓存的失效和命中的行为。可以看出，缓存命中的运行时间（约 30 个时钟周期）将显著小于失效的时间（近 300 个时钟周期）。这些侧信道信息也正是进行缓存攻击时利用的信息源，可以使用统计学的方法设置合理的时间阈值，根据缓存访问指令/数据的时间大于或者小于阈值的情况，来判断特定的指令/数据是从缓存中取得还是从主存中取得。

5.2.2 单台计算机攻击实验结果

前期准备实验完成后，使用本文提出的攻击模型，对目标模块进行了第一个分组最后一个字节的攻击实验，攻击的实验结果如图 5.2 所示。

实验时关闭了与实验无关的第三方进程，避免进程间的竞争给实验带来误差。目标

进程和攻击进程分别充当了通信活动中消息接收者和发送者的角色，在一轮攻击中，攻击进程向目标进程共发送 2^8 条消息，原因是攻击进程需要不断迭代随密文一起传送的初始化向量 ($temp_IV$) 的值达到篡改密文的目的，一个字节上 $temp_IV$ 可以变换的值有 2^8 个。图中横坐标为 $temp_IV$ 的值，纵坐标为攻击进程读取监测指令的时间。可以看到，当 $temp_IV$ 的值为 $0x92$ 的时候（其值会根据攻击实验中使用的明文/密钥/算法不同而有所变化），读取指令的时间大约为 112 个硬件时钟，大大小于使用其他 $temp_IV$ 时读取监测指令所需要的时间（约 460 个时钟）。这就说明，当 $temp_IV = 0x92$ 时，出现了一次缓存的命中，被监测的指令在被攻击进程二次读取前，已经被使用过了。结合之前对 Padding Oracle 产生过程的分析，可以断定这个值就是所需要的 Padding Oracle，即当初始化向量为 $0x92$ 的时候，解密后的结果其填充的合法性校验为真。

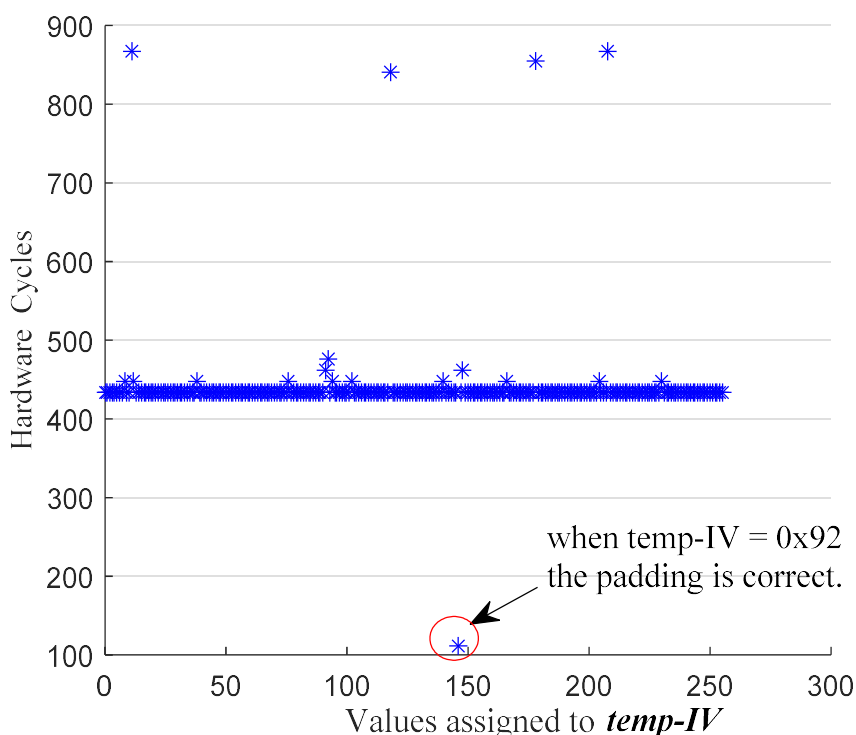


图 5.2 单台计算机上单字节攻击结果

使用数据恢复模块，将此时初始化向量的值 $0x92$ 带入，就可以得到解密后的分组中最后一个字节的值。如果希望得到整个一个分组的明文，则需要继续构造初始化向量，对倒数第二个字节进行攻击。该场景下的实验共进行了 200 次，有 99.5% 的攻击轮次成功获取到了 Padding Oracle 的结果，这也就意味着对单个字节的恢复成功率达到 99.5%。

5.2.3 跨虚拟机攻击实验结果

使用相同的攻击方法，在两个虚拟机上展开了攻击实验，虚拟化工具为 VMware 软件。目标进程和攻击进程在不同的虚拟机上，虚拟机同一台计算机上的硬件资源。实验

中使用的加解密算法依然为 AES-CBC-256，对密文第一个分组的最后一个字节的攻击结果，如下图 5.3 所示：

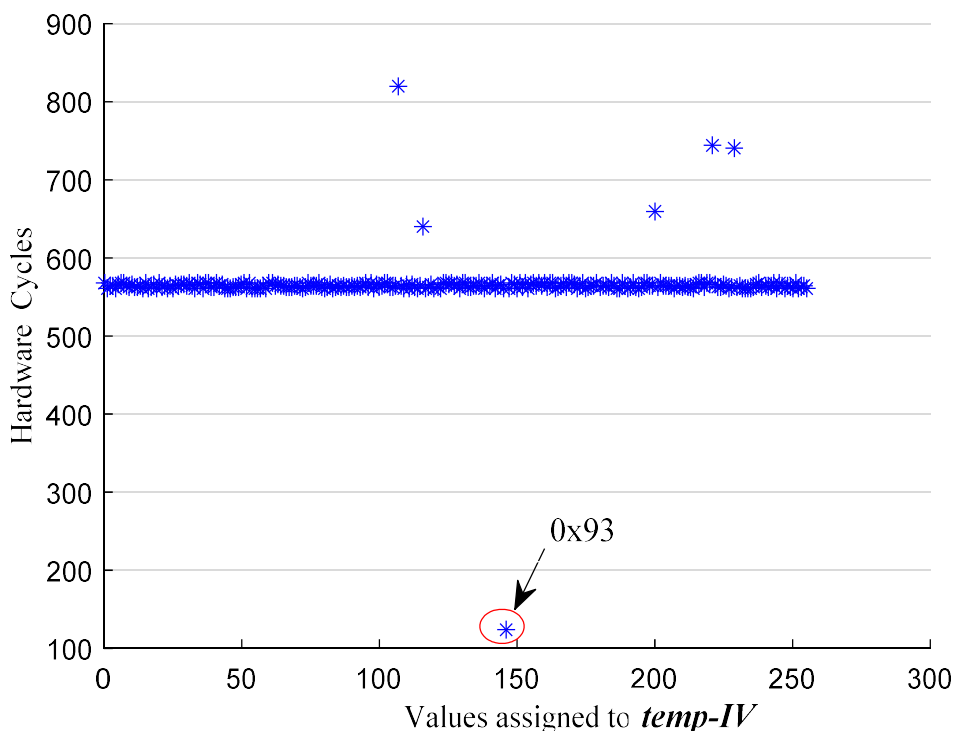


图 5.3 跨虚拟机单字节攻击结果

实验并没有得到预期的结果，因为按照算法的正常解密流程，只有在 $temp_IV = 0x92$ 的时候，才会出现缓存命中的现象，这个时候读取指令才会有较小的时钟周期。而在图 5.3 中，当 $temp_IV = 0x93$ 时，出现了缓存命中。导致这样结果的原因主要是进程间的竞争导致的。

在跨虚拟机的环境下，会有很多后台的进程运行，CPU 在执行目标的进程的加密操作时，可能会被这些不相干的进程打断，从而导致攻击进程监测时出现误差，即目标进程的解密操作并没有完全处于攻击进程的等待时间内，而是部分或者全部处于攻击进程的二次读取时间里。因此，如果要获取正确的 Padding Oracle，需要增加攻击的次数。

图 5.4 为不同轮次攻击单个密文分组最后一个字节时，获取到 Padding Oracle 的一个统计。其中横轴的数值代表攻击的次数，攻击进程总的提交消息的数量在每一轮中都是 2^8 次，所以如果是解密的消息数目作为攻击次数的统计量，则横轴上的数值还需要乘上 2^8 。从图中可以看到，只有当攻击轮次大于 4 次以上的时候，获取 Padding Oracle 的成功率才大于 0.5，而随着攻击轮次的增多，获取的成功率依次增加，当攻击轮次为 64 次的时候，其成功率已经达到 93.8%，这个时候攻击已经成功了，因为 $P(B)$ 只需要大于 50% 即可。

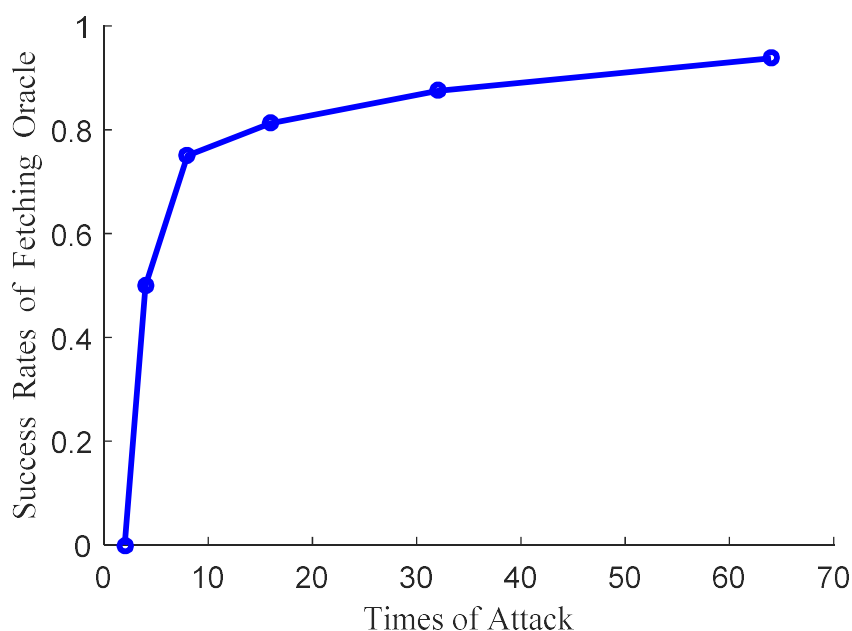


图 5.4 跨虚拟机单字节攻击成功率

实际攻击的时候,很难做到进行多轮次的攻击,这个时候需要构造精巧的密文分组,实现一次传输后的多轮次攻击。获取到 **Padding Oracle** 后,数据的恢复部分与单台计算机攻击实验的方法一致,可以得到分组密文最后一个字节所对应的明文。

5.3 实验结果分析

单台计算机上,最后一个字节的明文攻击实验表明,本文提出的攻击方法是有效的,恢复单个字节的成功达到了 99.5%。这个概率满足了不可区分实验 $\text{PrivK}_{A,\Pi}^{\text{CCA-POA}}(n)$ 中对获取 *Oracle* 的随机事件 *B* 的概率要求条件。同时,该结果也印证了第四章攻击模型效率分析中攻击效率平均值为 99% 的推断。之所以实际的攻击效果更好,一方面是 99% 的结果只是一个平均值,更重要的是,在进行攻击实验前,第三方的干扰进程已经被关闭了,实验中的噪声降到了最小。

跨虚拟机上的攻击实验进一步说明,不相干的进程对实验的攻击影响较大,这需要攻击者构造重复的密文进行多次提交,从而增大攻击成功的概率。如果实验过程中的噪声无法进一步降低,也可以在获取到实验数据进行统计分析达到减少噪声的目的。例如,可以引入人工神经网络 (Artificial Neural Networks) 来进一步降噪。神经网络通过模拟大脑的某些机理来实现某个方面的功能,通常都是对自然界某种算法或者函数的逼近。使用这种策略可以减少多次攻击的实验结果中出现的重复噪声,达到更为理想攻击效果。

目前国内外提出的基于 **Padding Oracle** 的攻击方法研究中,绝大多数都是利用了时间侧信道来获取 *Oracle*,这类方法存在的典型缺点就是噪声较大,如网络延迟就会带来严重的时间统计误差。在最新的构造方法中^[34],研究者也是通过不同次数的 HMAC 函

数调用引起的微小时间差,达到获得 *Oracle* 的目的,文中恢复两个字节的成功率可以达到 80%,对比本文提出的攻击模型,后者有更好的效果。如果密码算法的分组长度为 n (即有 n 个字节),单字节的攻击成功率为 p ,则整个分组的恢复成功率为 p^n ,因此明文分组(16 字节)的恢复成功率可以达到 92.3%,两个字节的恢复成功率约为 99%,远大于上述的 80%。从算法复杂度的角度分析,穷举单个分组的明文,其空间复杂度为 2^{128} ,而本文中的方法的复杂度为 $2^8 \times 16 = 2^{12}$,搜索的空间被大大降低。

5.4 抵抗攻击的措施

根据前面的分析,本文提出的攻击模型的两大核心是缓存攻击和 *Padding Oracle* 的漏洞,因此,针对该模型的防御方法,也须从这两方面考虑。

首先,缓存攻击利用了 x86 架构下的 *clflush* 指令从缓存中清除指令或者数据,因此,可以通过限制该指令的使用范围,达到防御这一类型缓存攻击的目的。增加进程的背景噪声也是减弱缓存攻击的另一种手段。显然,不同的进程共享了计算机的最后一级缓存,这种现象使得进程有了泄露敏感信息的风险,也是缓存攻击得以实现的物理基础。攻击进程在工作的时候,如果同时开启若干不相干的进程(噪声),就可以增加缓存攻击的难度:一方面 CPU 的进程调度策略可以打乱攻击进程使用固定时间访问缓存的时间链,另一方面进程数量的增加会使得缓存中数据的更新更加频繁。因此,在传输敏感数据的时候,多开几个应用,可以在一定程度上消除缓存攻击威胁。

从 *Padding Oracle* 漏洞的角度考虑,如何进一步地防止攻击者通过直接或者间接的方式获取到 *Oracle* 信息,是修补该漏洞的思路之一。目前的开源库厂商已经通过加密错误信息,时间平衡等手段避免了侧信道的出现,杜绝了绝大部分的基于 *Padding Oracle* 的攻击,但需要注意的是,对已有攻击手段的修复,可能会引入新的侧信道。此外,鉴于 AES-CBC 加密模式的脆弱性,在通信协议中使用新的加密策略成为了抵抗类 *Padding Oracle* 攻击的最好方法。譬如,可以使用 AES-GCM 和 AES-CCM 代替现有 AES-CBC,前者是一种结合了加密和消息验证的工作模式,可以带来高于后者的安全性和使用的简便性。

使用上述防御手段中的任何一种,可以破坏攻击流程中的某一环节,达到抵御攻击的目的。

5.5 本章小结

本章通过实验证实了本文提出的攻击模型的有效性。

实验前的一些准备措施是必不可少的,如合理设置缓存攻击每轮攻击的周期,如何避免预取技术给攻击带来误差等等。

在单台计算机上的攻击实验中,本文使用了两个进程模拟了攻击模型,结果显示最后一个字节的攻击有效率达到了 99.5%,在某些场景下,一个字节的泄露已经可以对整

个安全系统造成威胁，如银行交易中传输“是/否”消息的通信活动。在跨虚拟机的攻击实验中，由于背景噪声的存在，攻击的成功率大大降低，只能通过提交相同密文重复解密达到增大成功率的目的，这需要攻击者对密文进行复杂的构造。

在本章的最后，本文给出了针对这种攻击模型的若干防御措施。

第六章 总结与展望

6.1 研究总结

随着信息化进程的不断推进和移动互联网的迅猛发展,人们享受到了信息加快流通带来的各种便利,世界变得更小与更具活力。但与此同时,信息的加速流通,也带来了信息安全的隐患。无论是个人的隐私安全,还是商业公司中的机密数据,都对信息的安全传输提出了新的要求。密码学作为信息安全的基础,在安全性保障方面扮演着最为关键的角色。密码学的理论安全,通常是不等价于其在实际使用中的安全的,研究密码学的安全,不仅有着重要的理论意义,还可以为密码算法的具体实践提供可贵的指导思想。

本文针对对称加密算法中的 AES 算法进行了一系列的研究,通过引入缓存攻击这种侧信道获取信息的方式,提出了一种可以恢复出明文的攻击模型,在目前的研究中,基于 Padding Oracle 的攻击类型都已失效了,本模型的提出可以给研究者提供新的攻击思路。同时,文中也第一次给出了量化该类型攻击效率的方法,后文的实验证实了初步的效率估计。

攻击模型适用于所有使用 CBC 模式的对称加密算法,本文以 AES 算法阐述攻击原理。AES 算法是在当今世界使用最为广泛的对称加密算法,它在具体使用时有若干种工作模式。通过研究发现,可以利用其 CBC 模式下的安全性缺陷,对解密过程展开攻击。本文在 AES-CBC 模式存在的 CCA 安全缺陷的基础上,提出了一个利用缓存攻击获取加密活动时产生的 Padding Oracle,最后恢复明文的方法。这种方法弥补了在当前的 Padding Oracle 攻击中,无法获取 Oracle 信息这一缺陷。缓存攻击可以监测与 Oracle 的产生相关的指令,根据指令的执行与否,结合代码的执行逻辑(本文以 GnuTLS 开源库为例分析),推断出所需要的信息。

根据本文提出的攻击模型,在两种不同的场景下进行了攻击实验。实验场景一是在单台计算机下进行的,加密进程和攻击进程处于同一系统下。实验对使用 AES-CBC 加密算法加密后的密文进行了攻击,单字节的实验结果表明,明文恢复的成功率达到了 99.5%,这个结果比目前该领域中最新攻击方法的效率(80%)高很多。可以通过构造密文,同时设计流程更为复杂的攻击进程,达到恢复整个分组明文的目的。实验场景二是模拟了云平台下不同虚拟机在攻击模型下的表现。分别让攻击进程和目标进程工作在了不同的虚拟机之下,然后使用了相同的攻击方法进行明文获取。实验结果表明,该场景下攻击成功率受不相干进程的干扰较大,需要反复解密同一密文,达到提高攻击有效性的目的。

上述的两个实验场景中,本文使用了一些简化的手段来验证实验的结果。如为了使得 Padding Oracle 的获取更加精确,避免无关数据的产生,省略了通信双方的握手环节;

在跨虚拟机的攻击实验中，通过人为设置两个虚拟机在同一台计算机上，避免了“共驻((co-location))”问题的出现。因此，本章的两个实验只是对真实环境下攻击的一个模拟，实际的攻击中，攻击者在此攻击的基础上，需要使用更加具有深度的策略。例如，构造精妙的数据包、使用多个缓存攻击监测点、从获取的数据中提取有用信息等。一次实际的攻击，是一个系统性的工作，虽然上述两个实验场景是一种模拟，但是其攻击模型的基础没有改变，其思路可以为攻击者提供借鉴，攻击者可以利用其他的手段丰富这个模型，实现更高的攻击精度和更大的适用范围。

本文提出的攻击模型是建立在 CCA 安全缺陷上的，这种缺陷并不是算法本身固有的，而是 AES 算法在实际的使用过程中，不同的工作模式引入的负面作用。这论证了密码学中的一个观点：密码学的设计安全性并不等于其使用安全性。后文中的两个实验，进一步地佐证了这一点。因此，如何更有效地利用好各种优秀的密码算法，避免不当的实践带来的风险，是研究者及各大密码算法开源库厂商需要深切关注的要点，本文的这些内容也许会提供一些想法和思考。

6.2 研究展望

本文是作者在攻读硕士研究生期间做的工作，结合本文研究工作的现状及目前国内学者在密码学侧信道安全方面的研究，下一步可以在如下几个方面展开深入的研究。

1，云平台下的跨虚拟机研究。本文的实验只是一个简化的模型验证，实际工作的云是非常复杂的，如何定位目标虚拟机，如何将攻击虚拟机与目标虚拟机“共驻”，如何解决虚拟机之间的环境噪声，这些都是进一步研究的方向，同时也有着重要的实际意义。

2，攻击模型在不同平台下的迁移。本文的攻击方法详述，是以 GnuTLS 开源库为例的，作者在研究过程中发现，其他的一些开源库，如 PolarSSL 和 CyaSSL，存在同样的安全隐患。对其进行研究，可以为开源库的开发人员，提供有价值的参考意见，同时不同的研究场景，也会带在研究方向上带来新的机遇。

3，攻击模型在其他领域的使用。缓存攻击是一个有力的侧信道攻击方式，它利用了计算机的硬件特征，提取目标进程的敏感信息。本文提出的基于缓存攻击的模型，也可适用于其它不同的场景，如 SM2 加密算法中，通过监测与密钥相关的模乘指令，可以得到部分密钥信息。进一步地，只要是敏感信息的处理，表现在了缓存的行为特征上，攻击模型都会有用武之地，因此，诸如数据库，浏览器 cookie 等这样的敏感地带，都可以成为验证攻击有效性的目标。

鉴于作者水平有限，论文肯定还存在不足之处，敬请各位评审专家不吝指正。

参考文献

1. 冯登国. 密码分析学[M]. 清华大学出版社, 2000.
2. 周永彬,徐秋亮. 侧信道攻击理论与技术. 中国密码学发展报告 2008. 2008, pp191-204
3. Kocher P C. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems[C]//Advances in Cryptology—CRYPTO'96. Springer Berlin Heidelberg, 1996: 104-113.
4. Kocher P, Jaffe J, Jun B. Differential power analysis[C]//Advances in Cryptology—CRYPTO'99. Springer Berlin Heidelberg, 1999: 388-397.
5. Page. The Heartbleed Bug. <http://heartbleed.com/>.
6. Percival C. Cache missing for fun and profit[J]. 2005.
7. Tiri K, Aciçmez O, Neve M, et al. An analytical model for time-driven cache attacks[C]//Fast Software Encryption. Springer Berlin Heidelberg, 2007: 399-413.
8. Kelsey J, Schneier B, Wagner D, et al. Side channel cryptanalysis of product ciphers[M]//Computer Security—ESORICS 98. Springer Berlin Heidelberg, 1998: 97-110.
9. Page D. Theoretical Use of Cache Memory as a Cryptanalytic Side-Channel[J]. IACR Cryptology ePrint Archive, 2002, 2002: 169.
10. Tsunoo Y, Saito T, Suzaki T, et al. Cryptanalysis of DES implemented on computers with cache[M]//Cryptographic Hardware and Embedded Systems-CHES 2003. Springer Berlin Heidelberg, 2003: 62-76.
11. Tsunoo Y, Suzaki T, Saito T, et al. Timing attack on Camellia using cache delay in S-boxes[C]//Symposium on Cryptography and Information Security (in Japanese). 2003.
12. Bernstein D J. Cache-timing attacks on AES[J]. 2005.
13. Bertoni G, Zaccaria V, Breveglieri L, et al. AES power attack based on induced cache miss and countermeasure[C]//Information Technology: Coding and Computing, 2005. ITCC 2005. International Conference on. IEEE, 2005, 1: 586-591.
14. Neve M, Seifert J P. Advances on access-driven cache attacks on AES[C]//Selected Areas in Cryptography. Springer Berlin Heidelberg, 2007: 147-162.
15. Aciçmez O. Yet another microarchitectural attack: exploiting I-cache[C]//Proceedings of the 2007 ACM workshop on Computer security architecture. ACM, 2007: 11-18.
16. Zhang Y, Juels A, Reiter M K, et al. Cross-VM side channels and their use to extract private keys[C]//Proceedings of the 2012 ACM conference on Computer and communications security. ACM, 2012: 305-316.
17. Yarom Y, Benger N. Recovering OpenSSL ECDSA Nonces Using the FLUSH+ RELOAD Cache Side-channel Attack[J]. IACR Cryptology ePrint Archive, 2014, 2014: 140.
18. Benger N, van de Pol J, Smart N P, et al. "Ooh Aah... Just a Little Bit": A small amount of side channel can go a long way[J]. IACR Cryptology ePrint Archive, 2014, 2014: 161.
19. Yarom Y, Falkner K E. Flush+ Reload: a High Resolution, Low Noise, L3 Cache Side-Channel Attack[J]. IACR Cryptology ePrint Archive, 2013, 2013: 448.

20. Zhang Y, Juels A, Reiter M K, et al. Cross-Tenant Side-Channel Attacks in PaaS Clouds[C]//Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2014: 990-1003.
21. Vaudenay S. Security Flaws Induced by CBC Padding—Applications to SSL, IPSEC, WTLS..[C]//Advances in Cryptology—EUROCRYPT 2002. Springer Berlin Heidelberg, 2002: 534-545.
22. Canvel B, Hiltgen A, Vaudenay S, et al. Password interception in a SSL/TLS channel[M]//Advances in Cryptology-Crypto 2003. Springer Berlin Heidelberg, 2003: 583-599.
23. Bardou R, Focardi R, Kawamoto Y, et al. Efficient padding oracle attacks on cryptographic hardware[M]//Advances in Cryptology—CRYPTO 2012. Springer Berlin Heidelberg, 2012: 608-625.
24. Paterson K G, Yau A. Padding oracle attacks on the ISO CBC mode encryption standard[M]//Topics in Cryptology—CT-RSA 2004. Springer Berlin Heidelberg, 2004: 305-323.
25. Rizzo J, Duong T. Practical Padding Oracle Attacks[C]//WOOT. 2010.
26. Alfardan N, Paterson K G. Plaintext-recovery attacks against datagram TLS[C]//Network and Distributed System Security Symposium (NDSS 2012). 2012.
27. J. Black and H. Urtubia: Side-Channel Attacks on Symmetric Encryption Schemes The Case for Authenticated Encryption. In Proc. of 11th USENIX Security Symposium. USENIX (2002):
28. Goldwasser S, Micali S. Probabilistic encryption[J]. Journal of computer and system sciences, 1984, 28(2): 270-299.
29. Bellare M, Namprempre C. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm[M]//Advances in Cryptology—ASIACRYPT 2000. Springer Berlin Heidelberg, 2000: 531-545.
30. Dolev D, Dwork C, Naor M. Non-malleable cryptography[C]//SIAM Journal on Computing. 1998.
31. Xu Y, Bailey M, Jahanian F, et al. An exploration of L2 cache covert channels in virtualized environments[C]//Proceedings of the 3rd ACM workshop on Cloud computing security workshop. ACM, 2011: 29-40.
32. Tsafirir D, Etsion Y, Feitelson D G. Secretly Monopolizing the CPU Without Superuser Privileges[C]//USENIX security. 2007, 7: 1-18.
33. CIPRESSO, T., AND STAMP, M. Software reverse engineering. In Handbook of Information and Communication Security, P. Stavroulakis and M. Stamp, Eds. Springer, 2010, ch. 31, pp. 659–696.
34. Al Fardan N J, Paterson K G. Lucky thirteen: Breaking the TLS and DTLS record protocols[C]//Security and Privacy (SP), 2013 IEEE Symposium on. IEEE, 2013: 526-540.
35. Katz J, Lindell Y. Introduction to modern cryptography[M]. CRC Press, 2014, 59-100.
36. Hennessy J L, Patterson D A. Computer architecture: a quantitative approach[M]. Elsevier,

- 2011, 2011:124.
37. Guo F, Solihin Y. An analytical model for cache replacement policy performance[C]//ACM SIGMETRICS Performance Evaluation Review. ACM, 2006, 34(1): 228-239.

致 谢

在此文完成之际，离我的硕士研究生结束也不远了，我希望把最诚挚的感谢献给每一个关心和帮助我的人。

首先需要感谢的是我的导师李慧云研究员，在这短短的三年里，我一直得到了导师悉心的引导和鼓励。李老师严谨的治学态度和在学术上敏锐的洞察力，着实让我钦佩不已。她为我创造了宽松的学术氛围，一直强调要我根据自身的兴趣去做学问，不断开拓自己的学术视野。我在学术中曾一度磕磕绊绊，是她引导我掌握科学的研究方法，点亮了前进的方向，不断地鼓励我战胜一个又一个的困难。除了在学业上的受益，李老师极具亲和力的品质也影响了我做人做事的态度。三年多的时间，李老师的言传身教让我终生难忘，这将是我今后学习和工作的宝贵财富。

感谢中国科学院大学以及深圳先进技术研究院。在中国科学院大学的集中学习阶段，我认识了很多良师益友，静谧的环境也让我静下心来在知识的海洋中徜徉。深圳技术研究院是我后面两年科研和生活的地方，这里的老师和同学给了我莫大的帮助。

感谢我的师兄师姐，和他们朝夕相处的日子里，我们一起讨论疑难问题，交流学术动向，让我收获到很多。他们是：邵翠萍师姐、凌建辉师兄、周剑斌师兄和杨灿师姐。我在实验室的同学也帮助了我很多，无论是在生活中，还是学术上，我们的互动频繁，同时还建立了深厚的友谊。他们是：周启航、谢宏峰、夏伟等。

我在研究生这三年中的其他同学和朋友，在此也一并感谢，谢谢你们。

感谢我的父母，二十多年来，他们为我提供了无忧无虑的生活环境，尊重我的每一个选择，默默付出，无私奉献，使得我今天可以完成学业。他们为了我做了最好的父母榜样。

最后，我感谢我的爷爷奶奶，两位老人虽然对我的研究不甚了解，却一直敦促我在学术上刻苦钻研，保持一颗谦逊的心，尊敬师长，团结同学，培养“温良恭俭让”的良好品性。他们的一言一行，让我一路走来倍感温暖。

